

Sistemas Operativos

Clase 15 – Locks

~~11~~/05/2026

12

Quiz semana 8 (open):

Tema: Clase 14

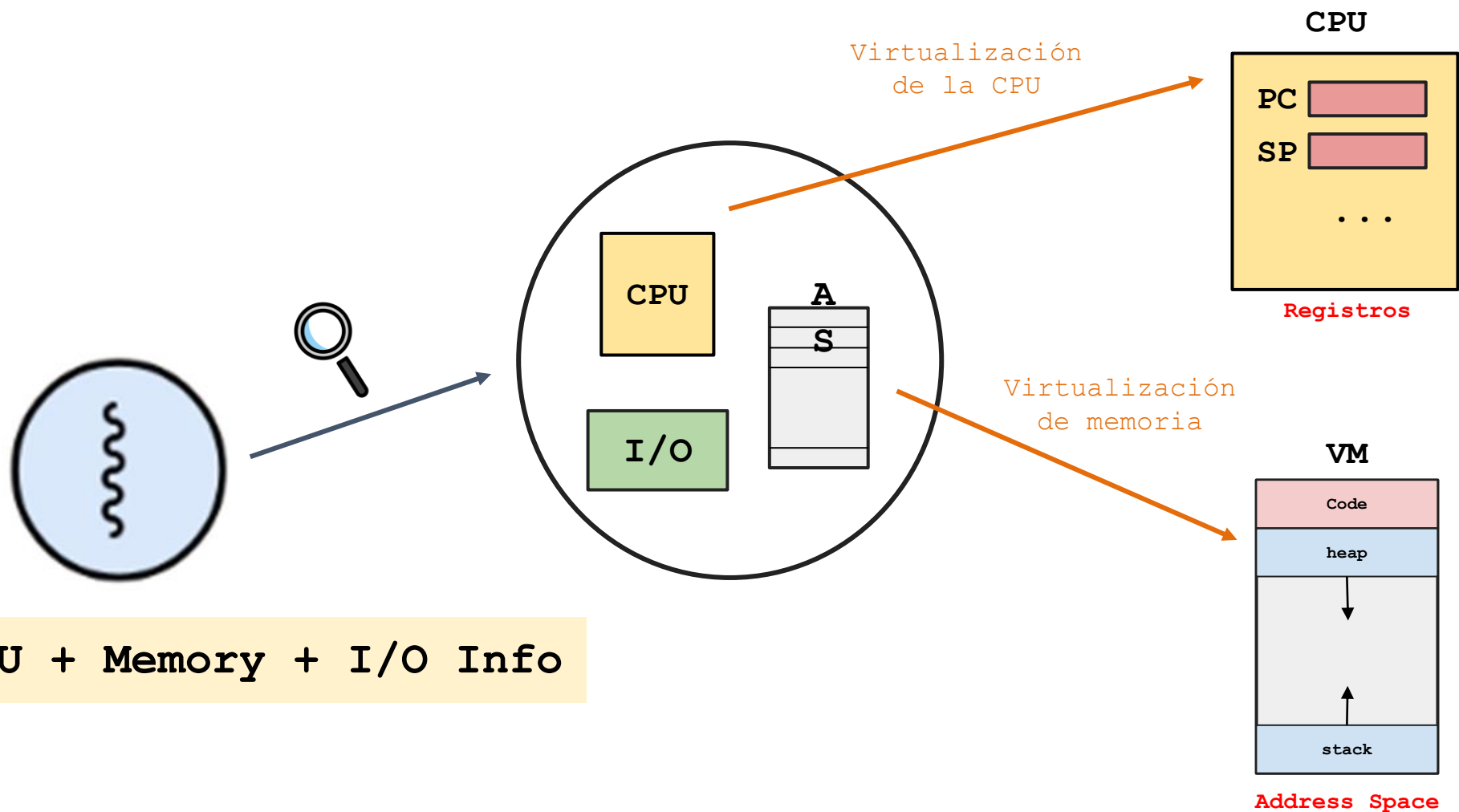
Quiz de autoevaluación: Semana 8

Abre: martes, 19 de mayo de 2026, 07:55

Cierra: martes, 26 de mayo de 2026, 23:59

Resumen virtualización

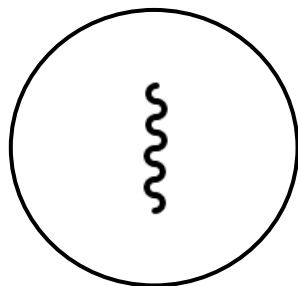
Procesos y virtualización



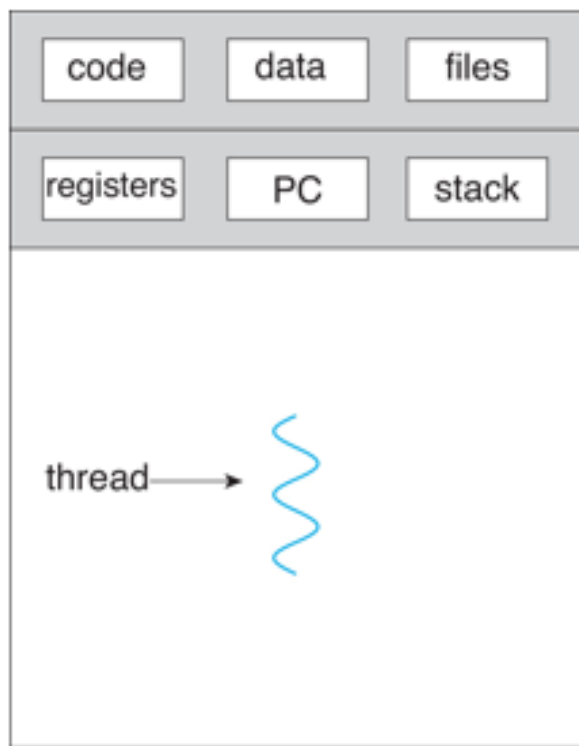
Proceso = CPU + Memory + I/O Info

Resumen virtualización

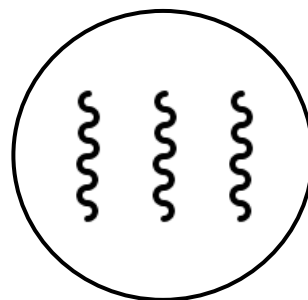
Procesos e hilos



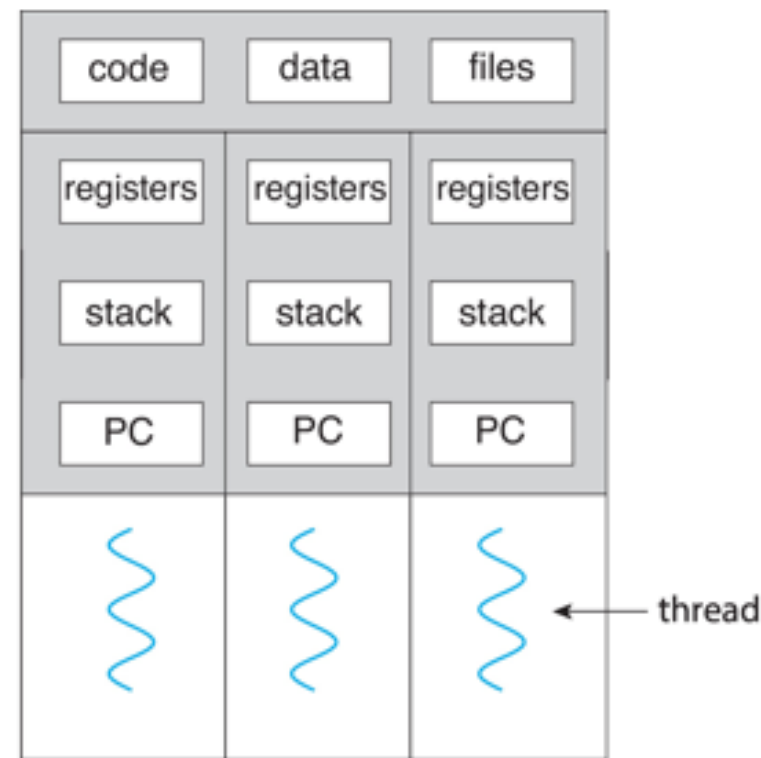
Un hilo de ejecución



single-threaded process



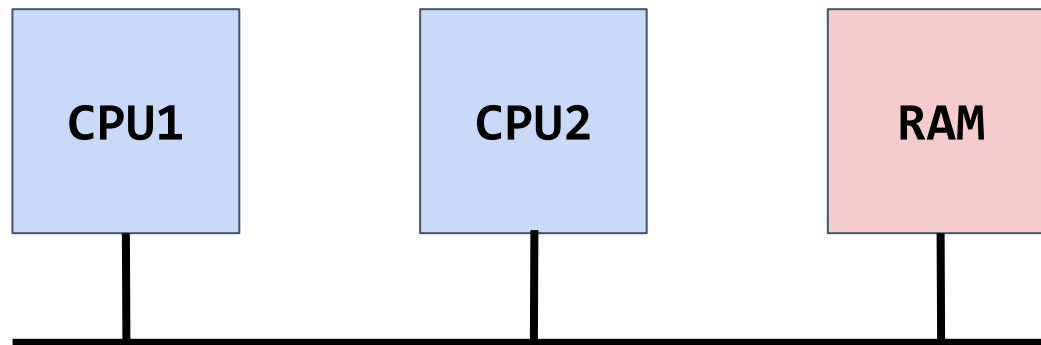
Múltiples hilos de ejecución



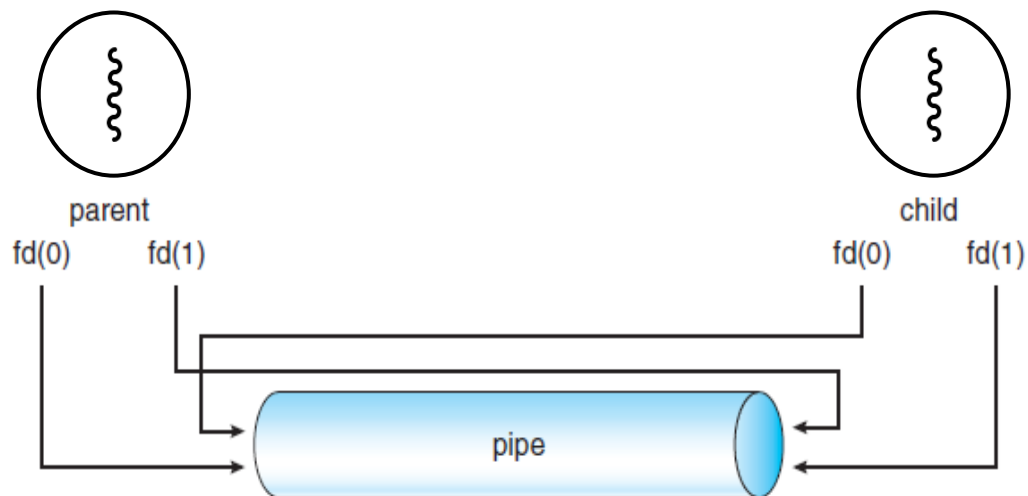
multithreaded process

Resumen virtualización

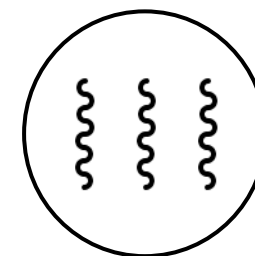
Aplicaciones y concurrencia



Procesos que se comunican entre sí

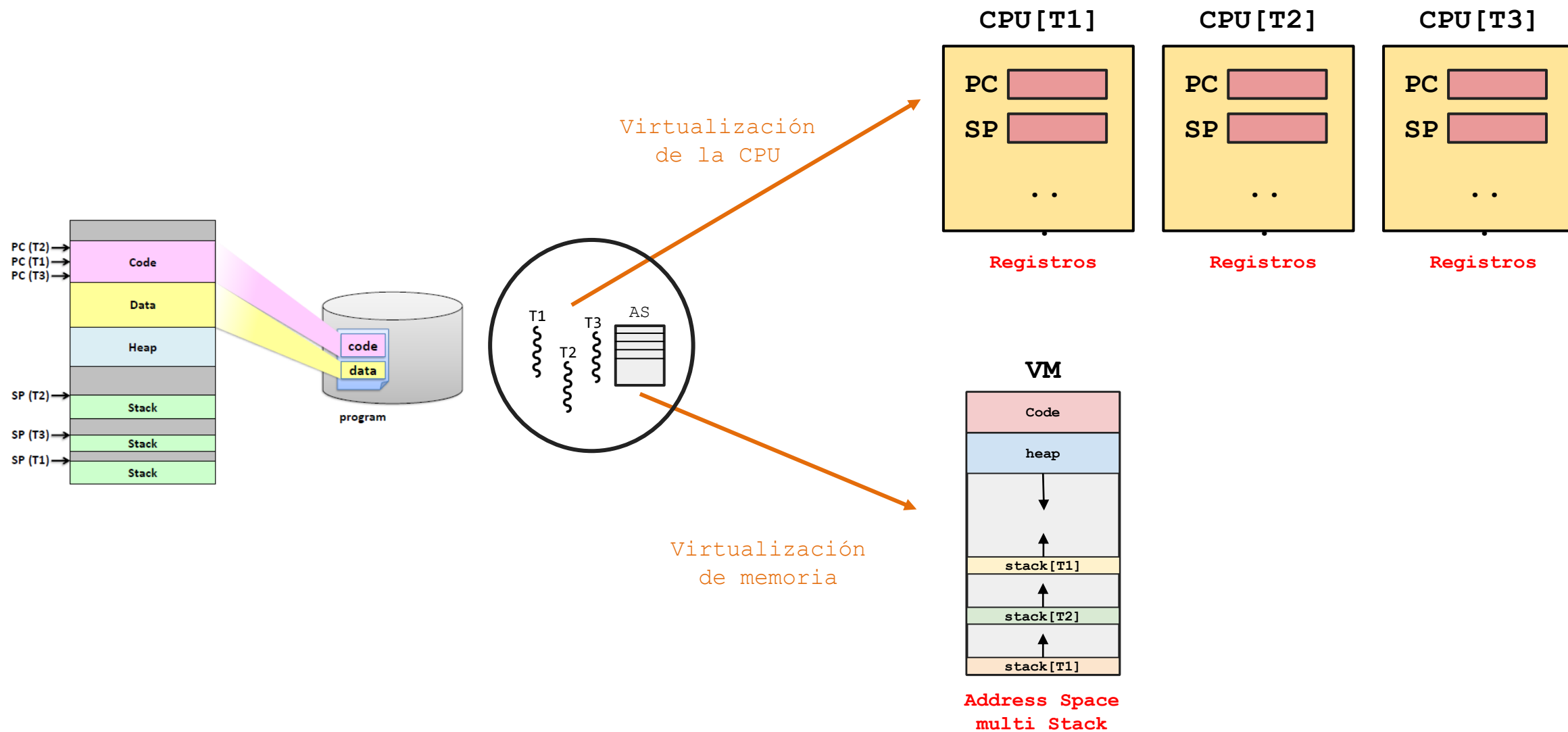


Procesos multihilo



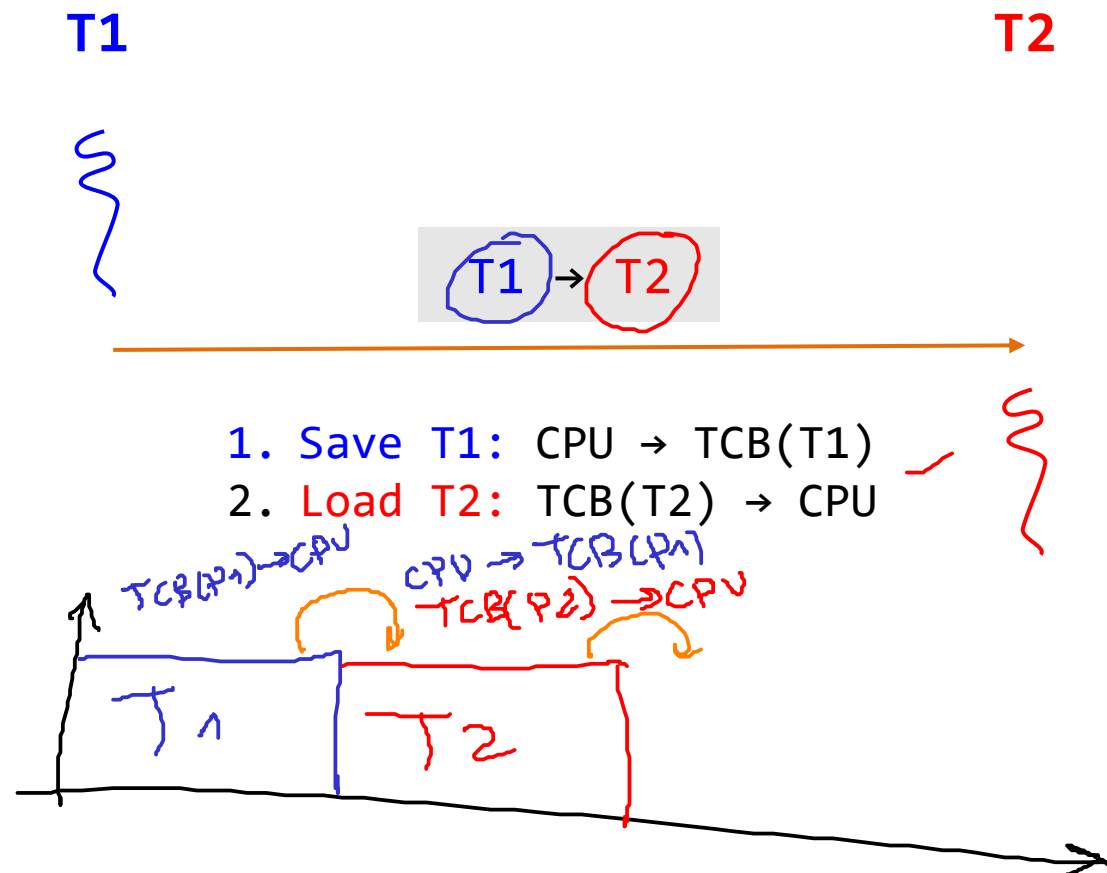
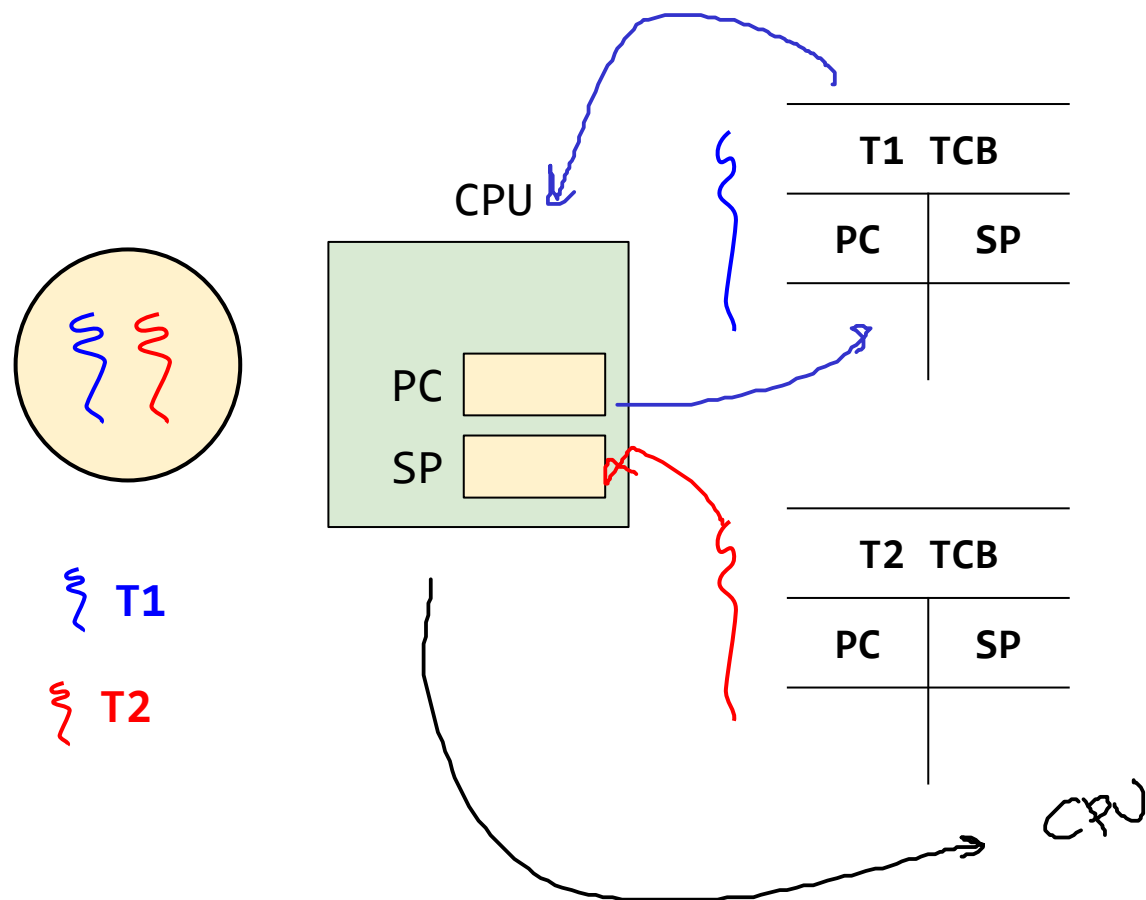
Resumen virtualización

Virtualización de Memoria



Resumen virtualización

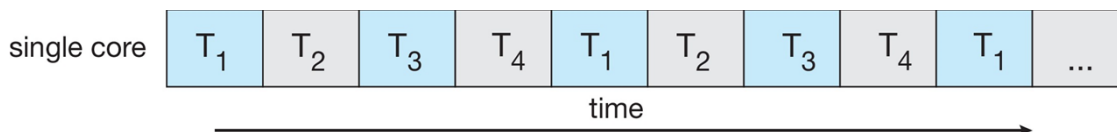
Virtualización de Memoria



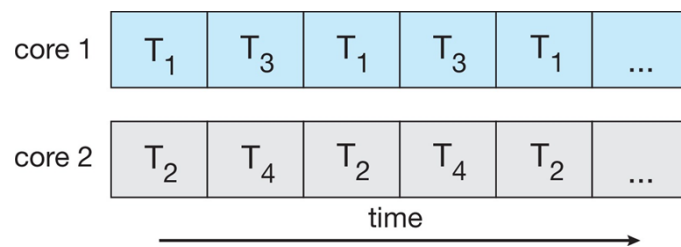
Resumen virtualización

Concurrencia .vs. Paralelismo

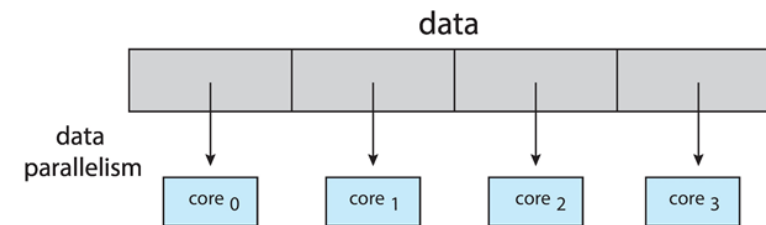
Concurrencia



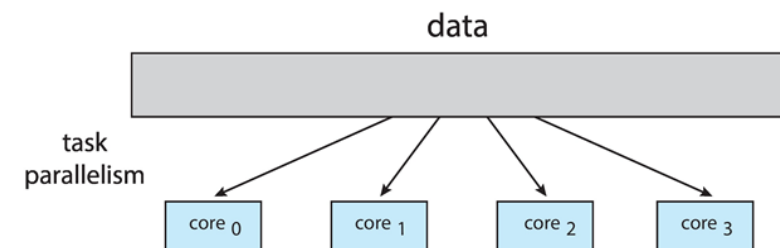
Paralelismo



Paralelismo
de datos

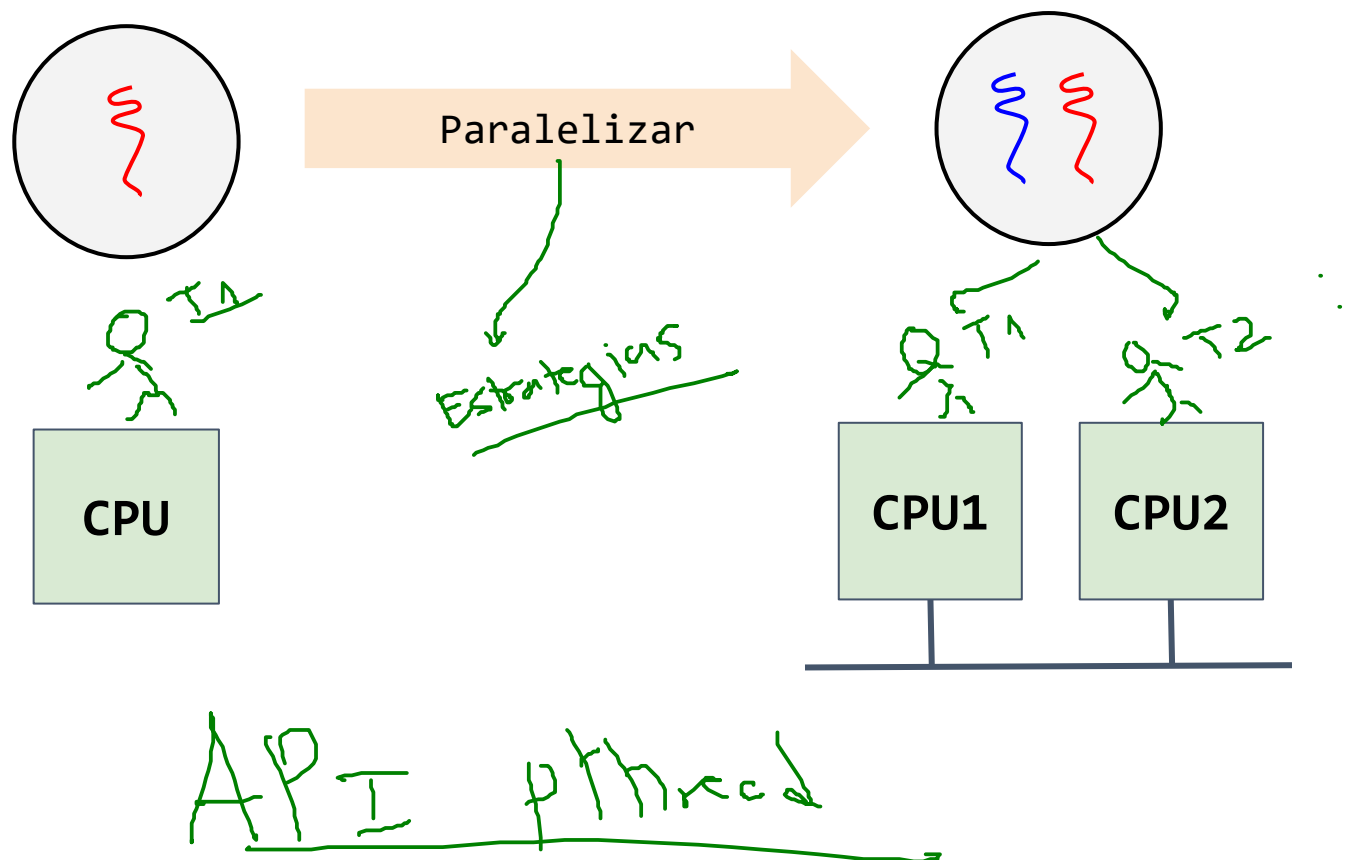


Paralelismo
de tareas



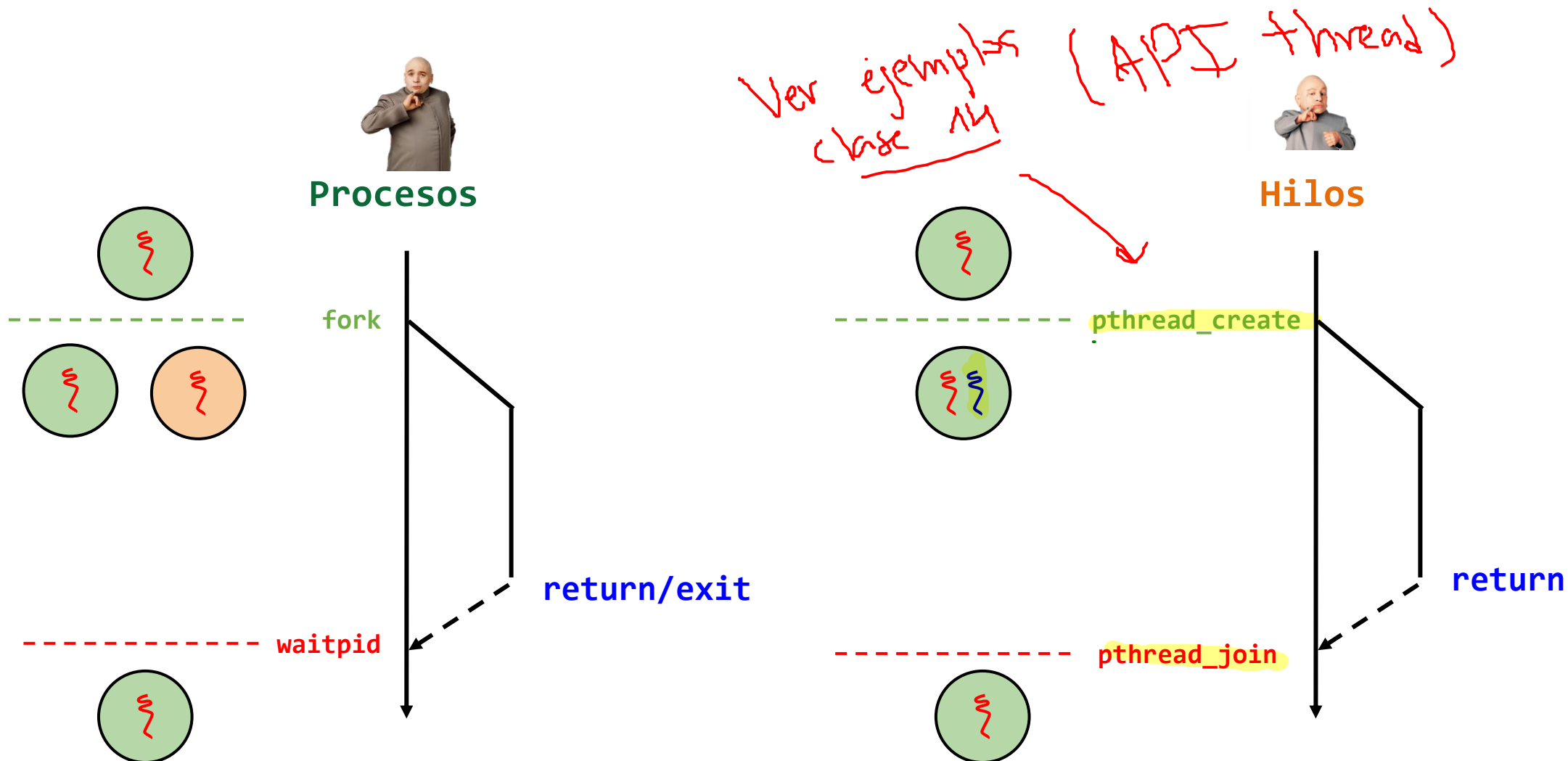
Resumen virtualización

Paralelizar código



Resumen virtualización

Comparación entre procesos e hilos



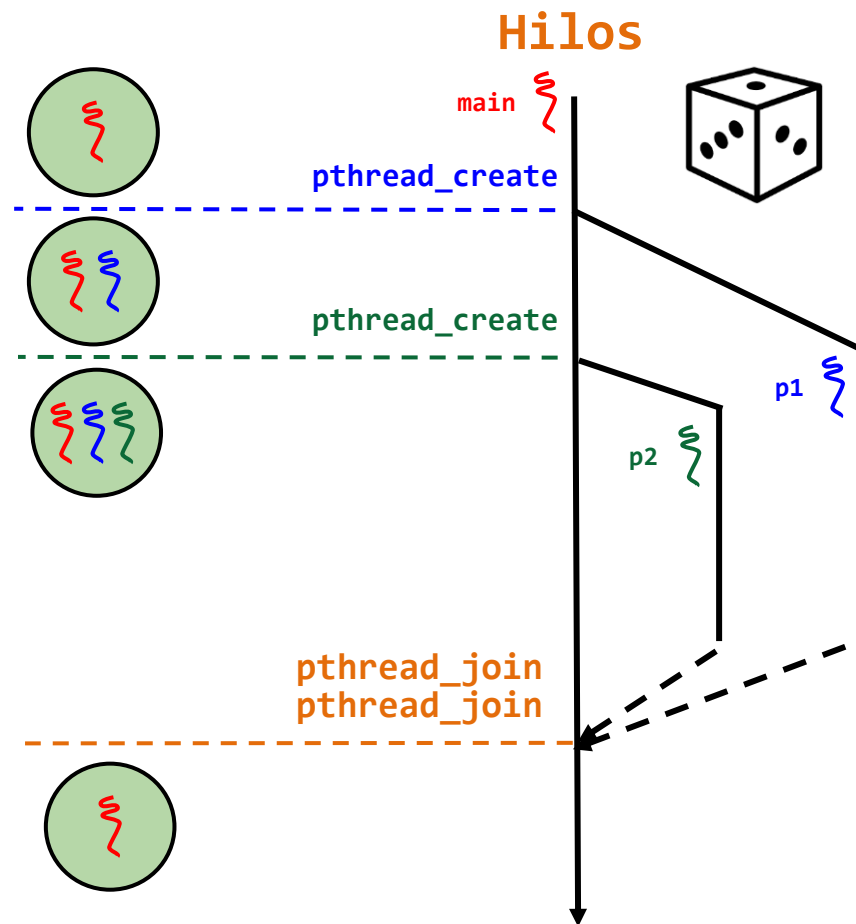
Resumen virtualización

Virtualización de Memoria - Virtual memory

tf.c

main

```
int main(int argc, char *argv[]) {  
    ...  
    pthread_t p1, p2;  
    printf("main: begin\n");  
    Pthread_create(&p1, NULL, mythread, "A");  
  
    Pthread_create(&p2, NULL, mythread, "B");  
  
    Pthread_join(p1, NULL);  
    Pthread_join(p2, NULL);  
    printf("main: end\n");  
    return 0;  
}
```



p1

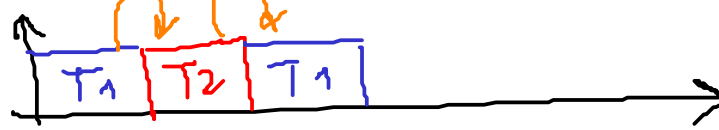
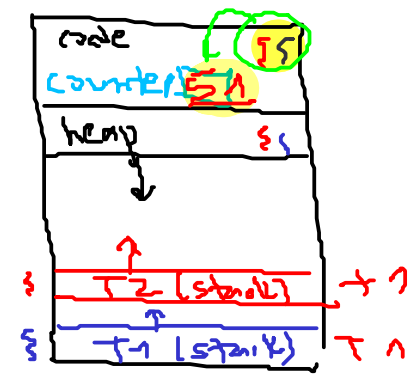
p2

```
void *mythread(void *arg) {  
    printf("%s\n", (char *) arg);  
    return NULL;  
}
```

Resumen virtualización

Virtualización de Memoria

(Race condition)



PC[T1]	PC[T2]
100	
105	
108	
	100
	105
	108
	113
113	

counter = counter + 1;

100
105
108

```
mov 0x8049a1c, %eax
add $0x1, %eax
mov %eax, 0x8049a1c
```

T1

PC	eax
108	51

T2

PC	eax
100	0

OS Thread 1

Thread 2

before critical section
mov 8049a1c, %eax
add \$0x1, %eax

interrupt
save T1
restore T2

```
mov 8049a1c, %eax
add $0x1, %eax
mov %eax, 8049a1c
```

interrupt
save T2
restore T1

(after instruction)

PC	eax	counter
100	0	50
105	50	50
108	51	50
100	0	50
105	50	50
108	51	50
113	51	51
108	51	51
113	51	51

CPU

eax	PC
51	108
50	105
51	108
50	105
51	108
50	105
51	108
50	105
51	108

counter 51 0x8049a1c = &counter

mov %eax, 8049a1c

Contextualización

Sección crítica

Es una **pieza de código** que accede a un recurso compartido, usualmente una variable o estructura de datos.

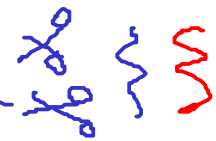


```
void *mythread(void *arg) {  
    char *letter = arg;  
    int i; // stack (private per thread)  
    printf("%s: begin [addr of i: %p]\n", letter, &i);  
    for (i = 0; i < max; i++) {  
        counter = counter + 1; // sección crítica  
    }  
    printf("%s: done\n", letter);  
    return NULL;  
}
```

counter = counter + 1; ⚡⚡



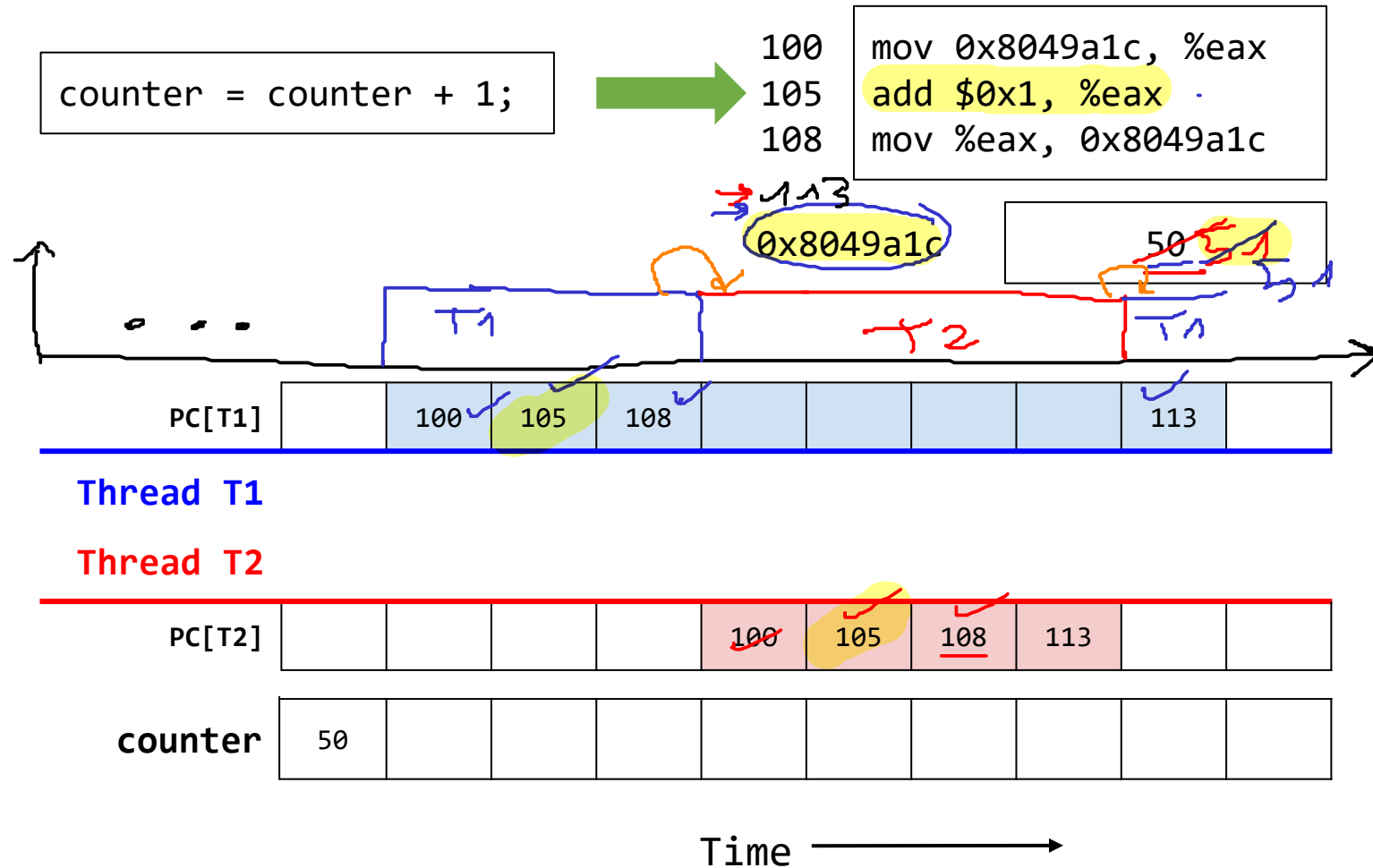
```
100 mov 0x8049a1c, %eax  
105 add $0x1, %eax  
108 mov %eax, 0x8049a1c
```



Contextualización

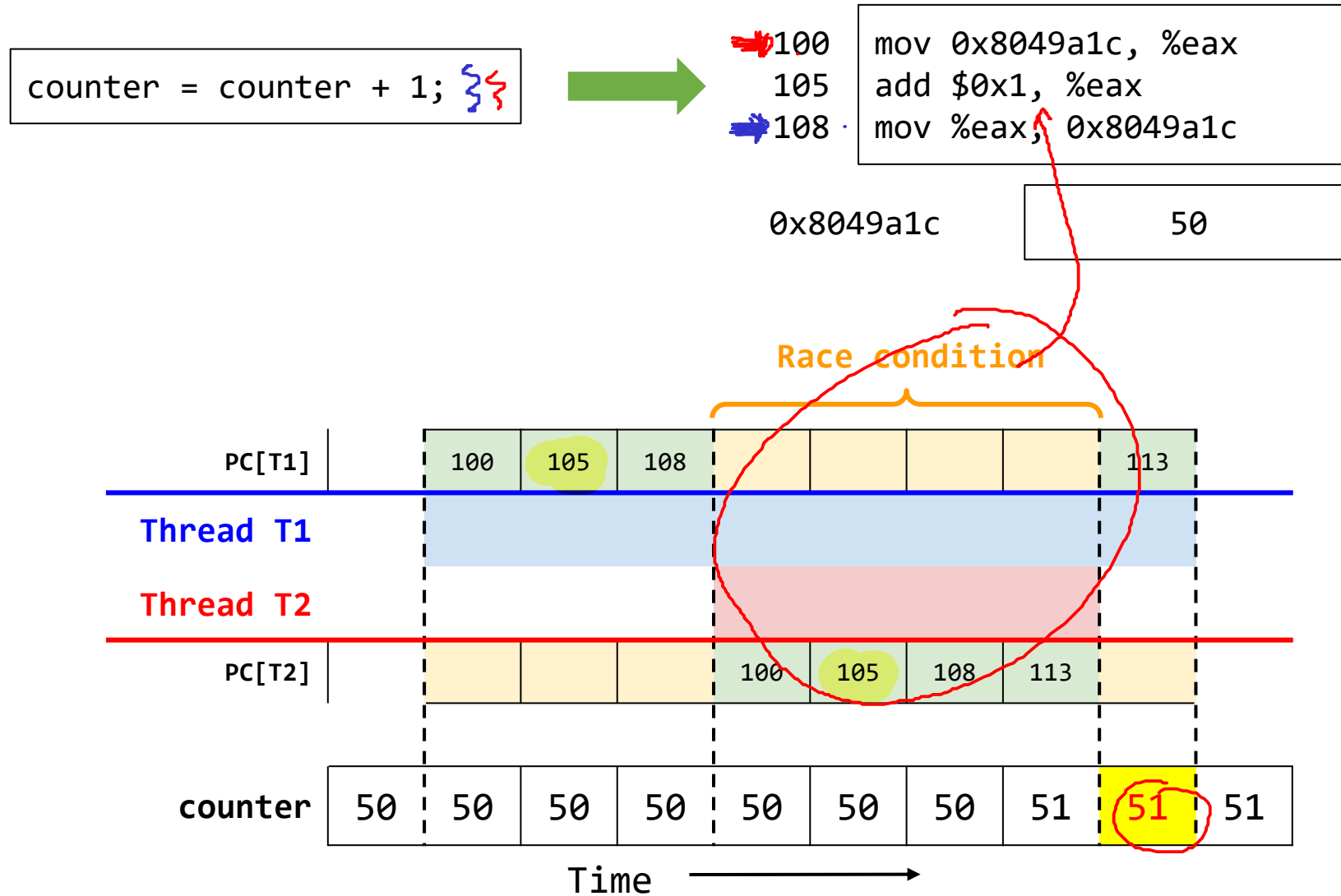
Race condition (condición de carrera)

CPU		T1		T2	
PC	%eax	PC	%eax	PC	%eax
?	?	100	0	100	0
100	50	108	51	113	51
105	50				
108	51				
100	50				
105	50				
108	51				
113	51				
108	51				
113	51				
108	51				
113	51				



Contextualización

Race condition (condición de carrera)



Contextualización

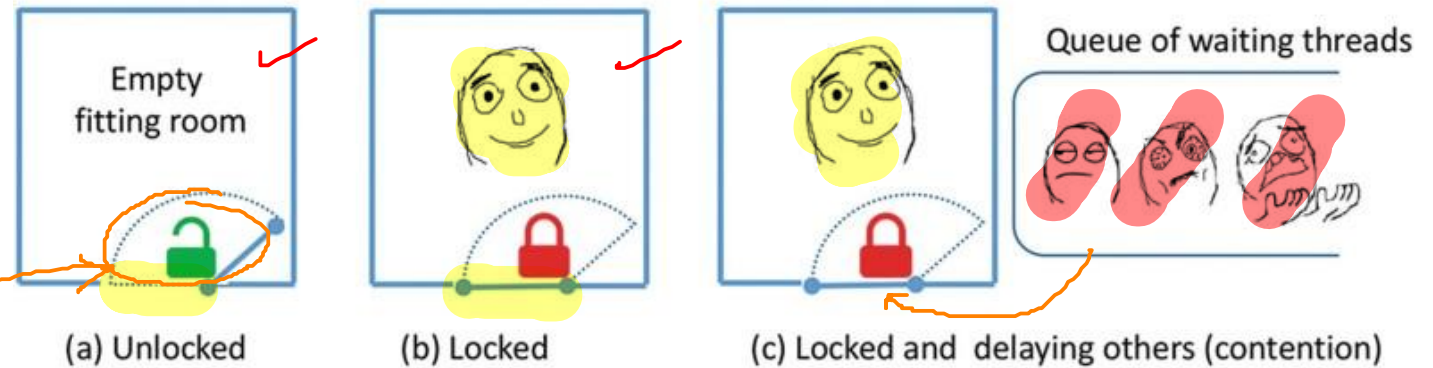
Exclusión mutua ✓

Garantía de que si un hilo se encuentra dentro de una sección crítica, ningún otro hilo puede acceder a esta.

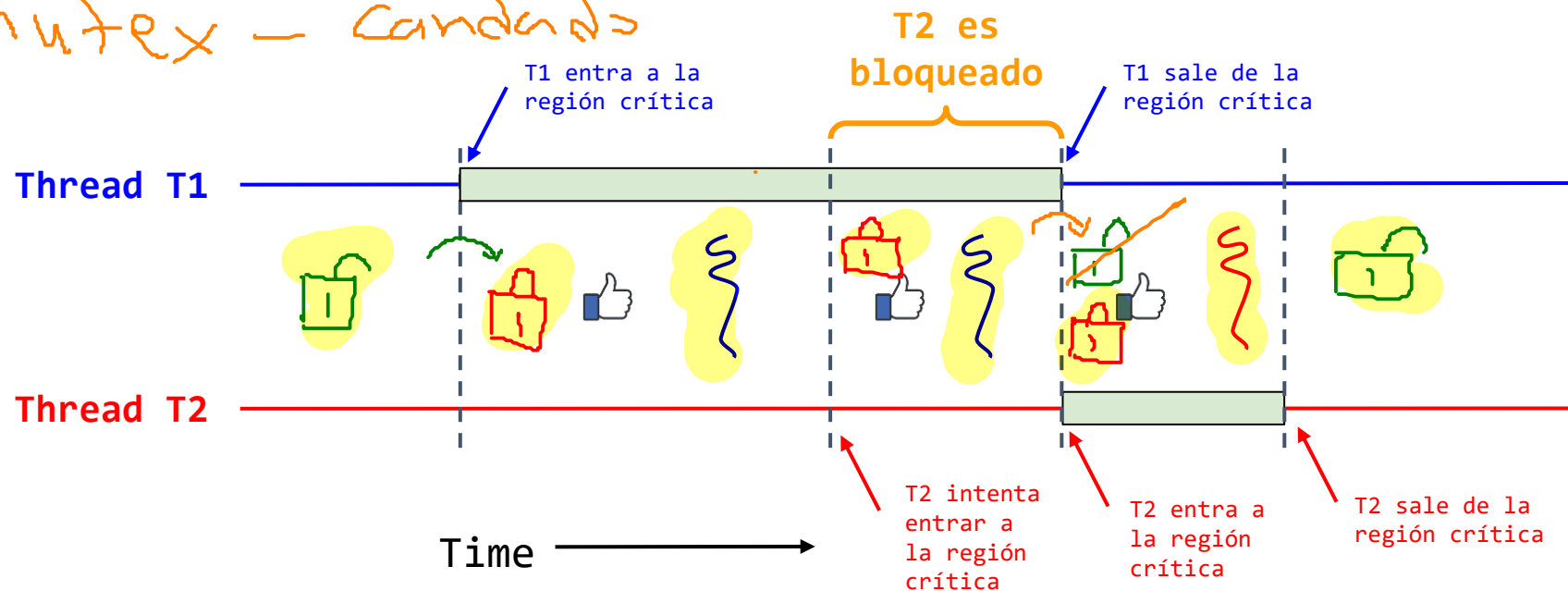
Next: 19/05/2026

Como hago para evitar Race Condition?

The fitting room analogy

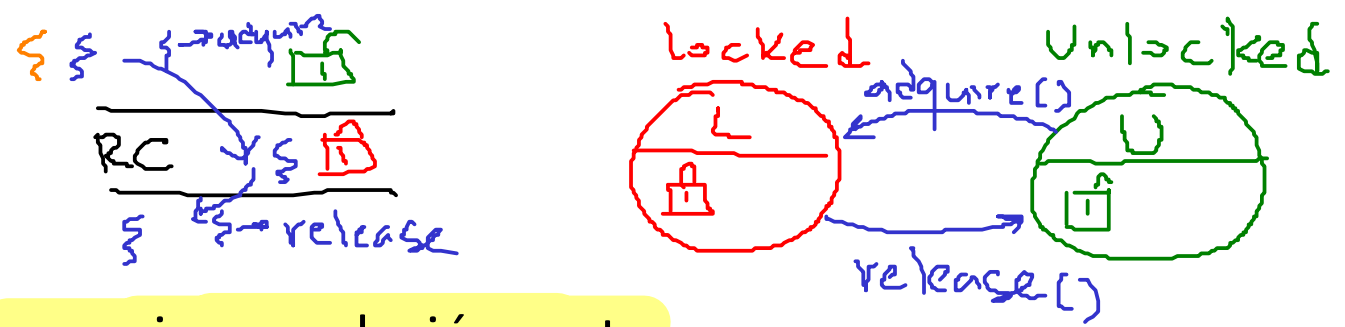


Lock — mutex — candado



Contextualización

Locks



Un lock es un objeto (en memoria) que proporciona exclusión mutua.

Estados



acquired (locked, held):

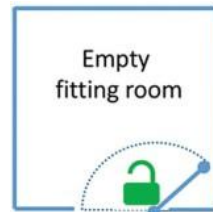
Indica que un hilo ha **adquirido** el lock y que este, se está ejecutando en la sección crítica



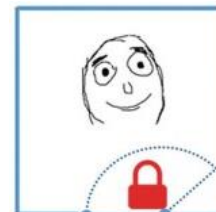
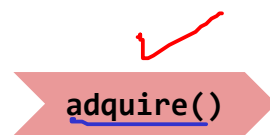
available (unlocked, free):

Indica que ningún hilo ha obtenido el lock.

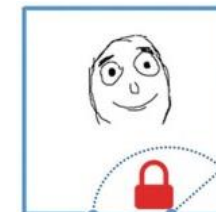
Operaciones



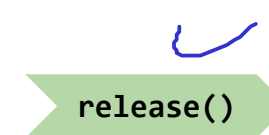
(a) Unlocked



(b) Locked



(b) Locked



(a) Unlocked

acquire(): espera hasta que un lock este libre, y cuando lo esta, lo toma.

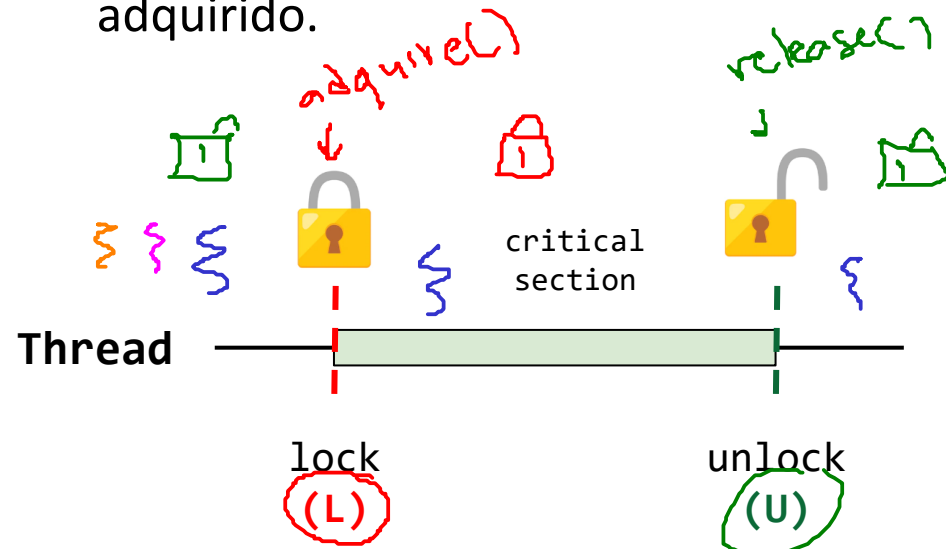
release(): libera el lock y permitiendo que cualquier hilo que está esperando por este para tomarlo.

Contextualización

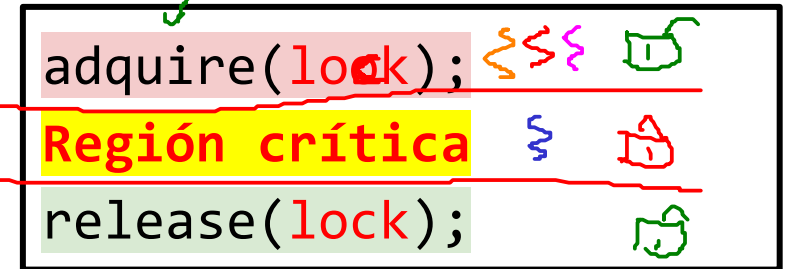
Usando Locks

Procedimiento:

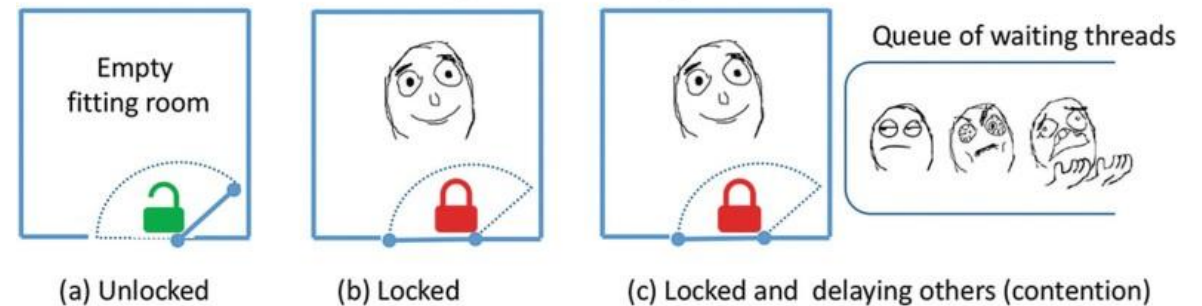
1. Inicialmente el **lock** está libre. ✓
2. Se llama **acquire()** antes de entrar a la región crítica. ✓
3. Se ejecuta el código de la **región crítica**. ✓
4. Se llama **release()** para liberar el **lock** previamente adquirido. ✓



Patrón de uso de locks
(Mutex = Exclusion mutua)
Solo puede haber un hilo



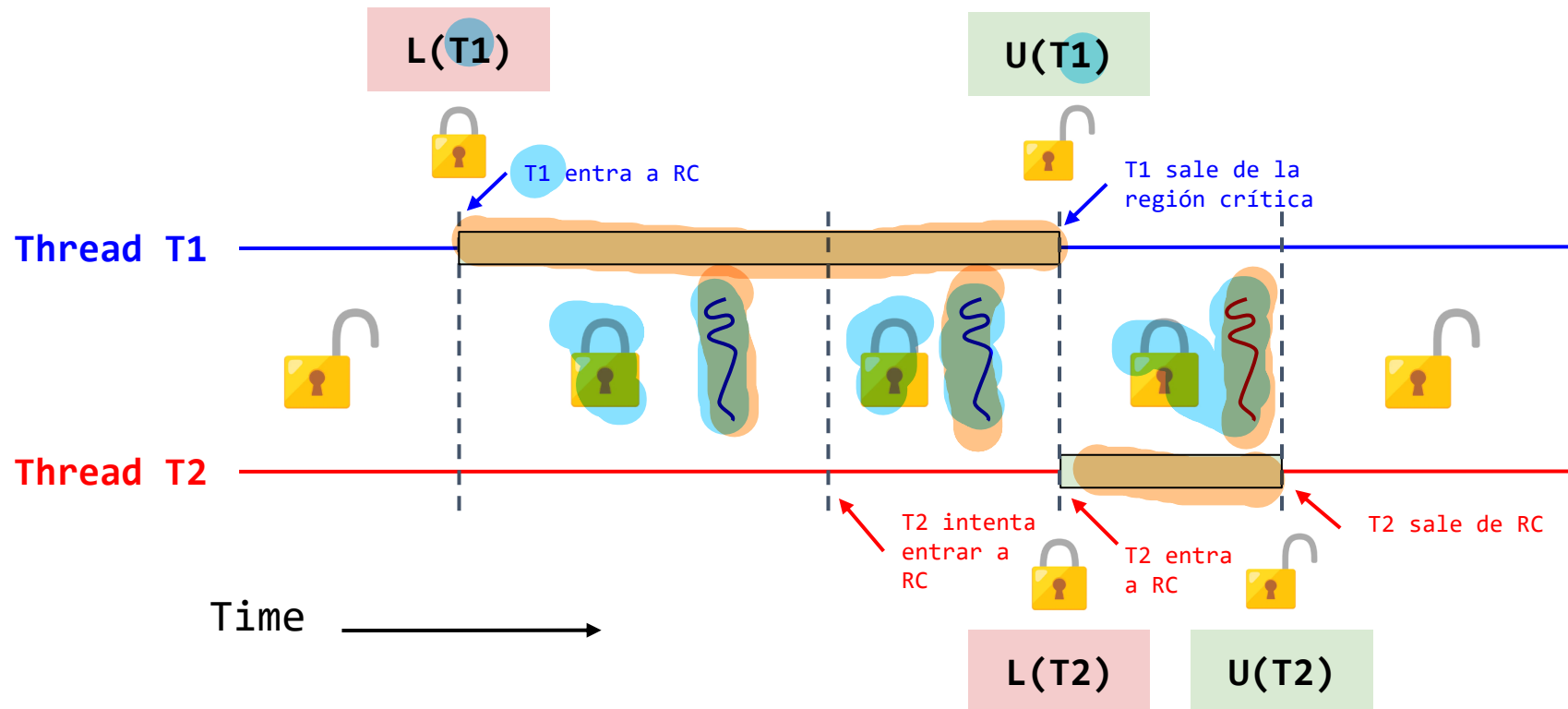
The fitting room analogy



Contextualización

```
acquire(lock);  
critical section;  
release(lock);
```

```
lock(&mutex);  
balance = balance + 1;  
unlock(&mutex);
```



Contextualización

Usando locks - Estrategias

Estrategia 1 - Coarse-grained

locking strategy: Un lock global se emplea cada vez que se accede a una región crítica (**Forma antigua**).

LC) —
Reg. Crit. —
UC) —



```
lock_t mutex; // some globally-allocated lock
               // 'mutex'

...
lock(&mutex);
balance = balance + 1;
unlock(&mutex);
```

Estrategia 2 - Fined-grained locking strategy: Se protegen diferentes secciones críticas empleando diferentes locks (**Posix**).



LC) —
Reg. Crit. —
UC) —

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

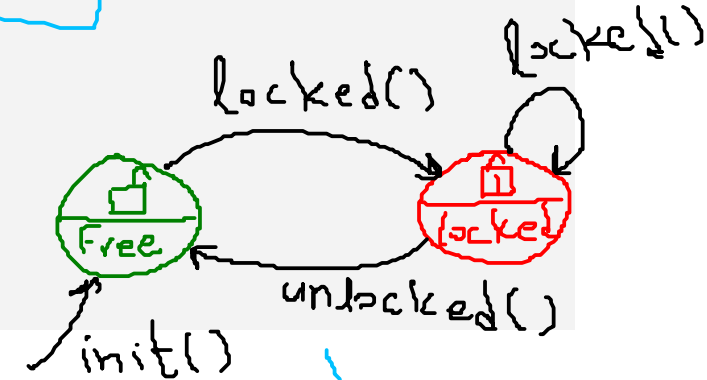
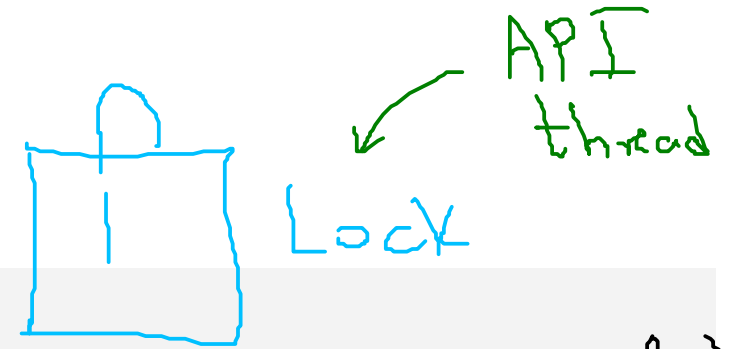
pthread_mutex_lock(&lock); // wrapper; exits on failure
balance = balance + 1;
pthread_mutex_unlock(&lock);
```

Contextualización

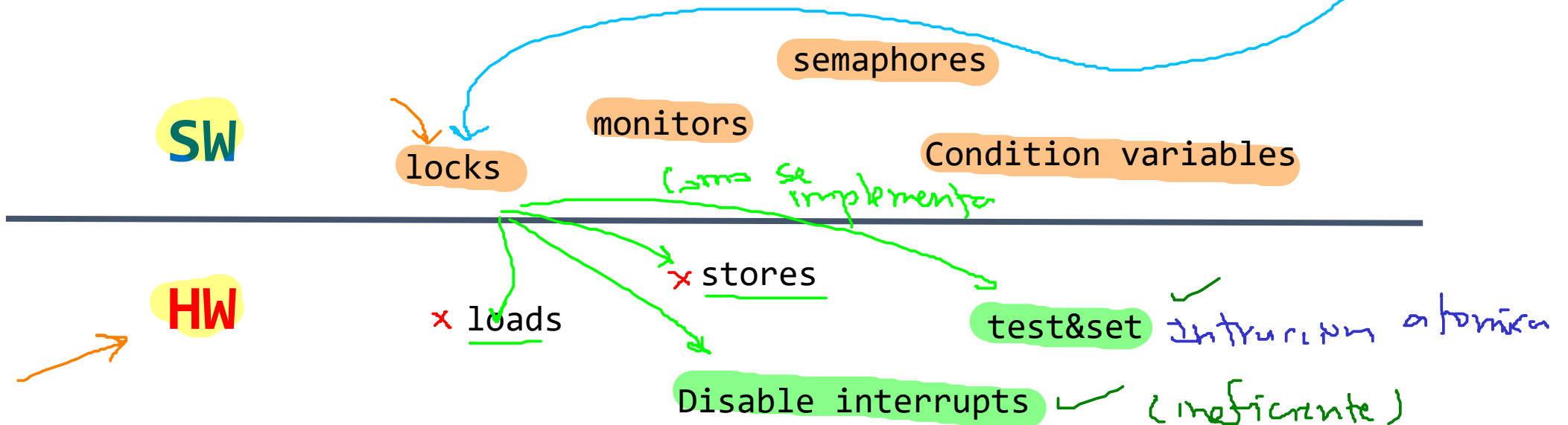
Implementación de locks

¿Cómo construir un lock?

- ¿Cómo construir un lock que sea eficiente?
- ¿Qué hardware es necesario (Primitivas del ISA, su uso, etc)?
- ¿Cuál es el soporte del SO (Qué necesitamos usar de este)?



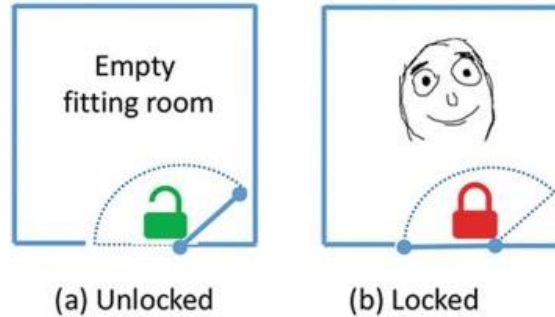
Objetivo: Obtener exclusión mutua con el menor costo posible.



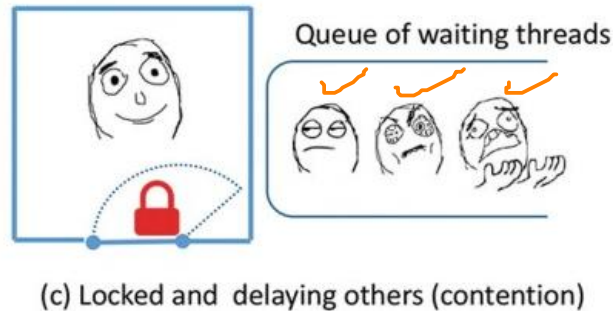
Evaluación de locks

Para evaluar la efectividad en la implementación de un lock se utilizan las siguientes métricas:

- ✓ ● **Mutual exclusion (Exclusión mutua)**: Sólo un hilo en la sección crítica.



- ✓ ● **Fairness (Equidad)**: Cada hilo tiene igual probabilidad de adquirir el lock.

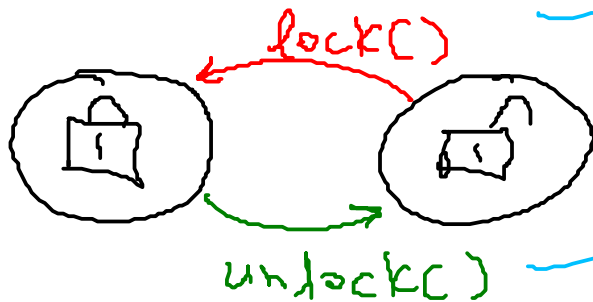


- ✓ ● **Performance (Desempeño)**: Relacionado con el time overhead (tiempo tomado en la adquisición/liberación con o sin competencia).

Implementación de locks

Implementación:

Determinación de cómo implementar las funciones **lock()** y **unlock()**.



```
void lock() {  
    // Instrucciones lock  
    // ...  
}  
  
void unlock() {  
    // Instrucciones lock  
    // ...  
}
```

Como las usinas!
esta clase

La siguiente tabla muestra algunas formas de implementar locks:

Algoritmos de solo software	Instrucciones atómicas
Algoritmo de Dekker (1962)	Test & Set ✓
Algoritmo de Petterson (1981)	Compare & Swap ✓
Algoritmo de Lamport para más de dos procesos (1974)	Load-Linked (LL) & Store-Conditional ✓
	Fetch & add ✓

CIAW-ISA)

Implementación de locks

Implementación mediante el control de interrupciones:

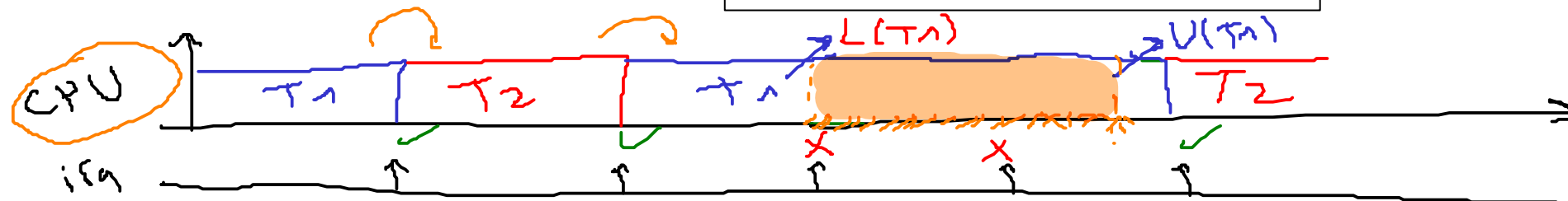
- Deshabilitar interrupciones al entrar a las regiones críticas.
 - Cuando se deshabilitan las interrupciones se bloquean eventos externos que podrían generar un cambio de contexto.
 - El código dentro de la sección crítica no será interrumpido.
 - No hay un estado asociado al lock.

```
void lock() {  
  ✗ DisableInterrupts();  
}
```

```
void unlock() {  
  EnableInterrupts();  
}
```

lock ([])
Region critica
{ }

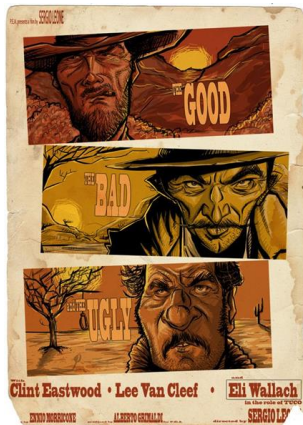
unlock ([])



Implementación de locks

Implementación mediante el control de interrupciones

```
void lock() {  
    DisableInterrupts();  
}  
  
void unlock() {  
    EnableInterrupts();  
}
```



Lo bueno	Lo malo
• Simple. • Útil para un sistema con un solo procesador.	• Uso inadecuado al abusar de la capacidad de habilitar y deshabilitar interrupciones. • Esta aproximación no trabaja con sistemas con varios procesadores. • Cuando se desactivan las interrupciones durante un periodo de tiempo largo, las interrupciones importantes pueden perderse. • Ineficiente: el código para habilitar y deshabilitar interrupciones es lento (respecto al código que se ejecuta normalmente) en procesadores modernos.

Implementación de locks

API
pthread.h

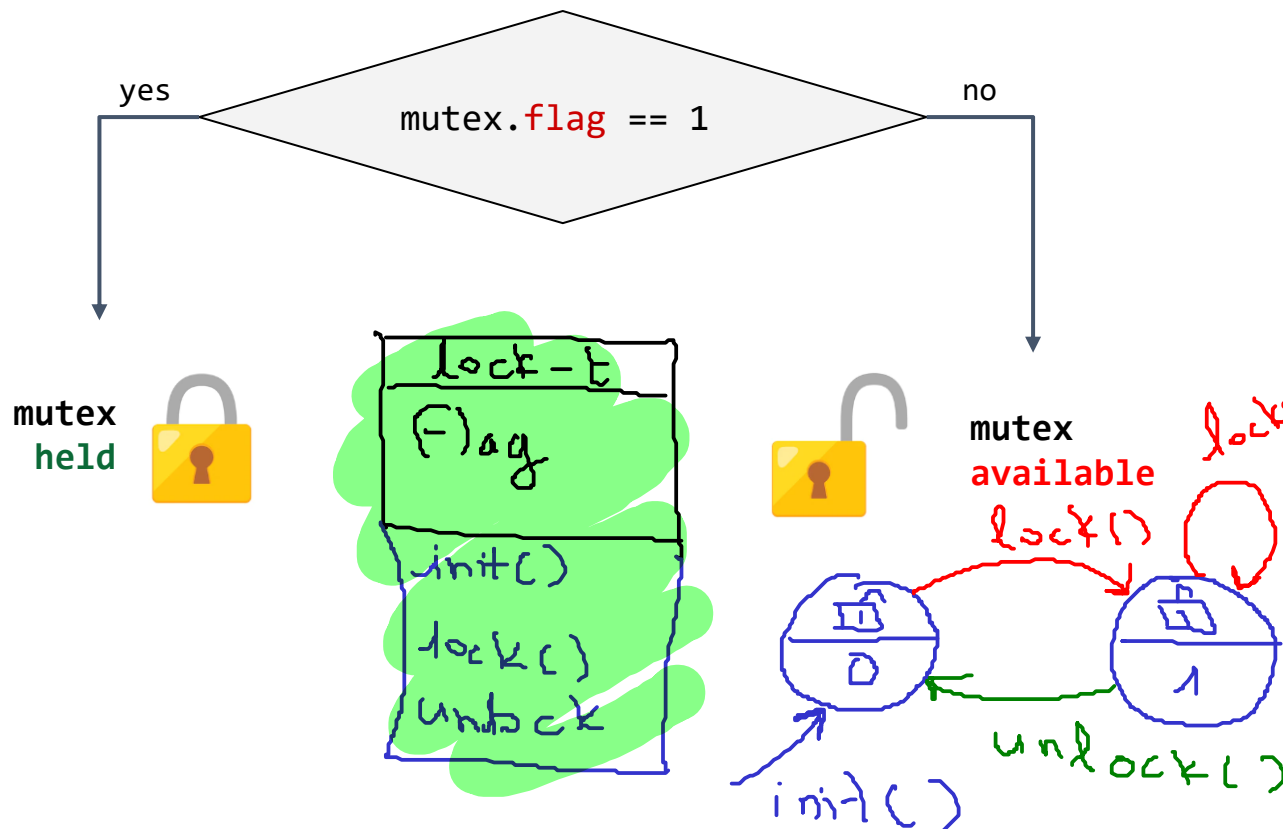
`init()` ✓
`lock()` ✓
`unlock()` ✓

Implementación mediante el uso de instrucciones Load/Store

pthread.c

En esta solución por software, se usa una **flag** para determinar el **estado**.

Problema



```
1 typedef struct __lock_t {
2     int flag;
3 } lock_t;
4
5 void init(lock_t *mutex) {
6     // 0 -> lock is available, 1 -> held
7     mutex->flag = 0;
8 }
9
10 void lock(lock_t *mutex) {
11     while (mutex->flag == 1) // TEST the flag
12         ; // spin-wait (do nothing)
13     mutex->flag = 1; // now SET it!
14 }
15
16 void unlock(lock_t *mutex) {
17     mutex->flag = 0;
18 }
```

Implementación de locks

Implementación mediante el uso de instrucciones Load/Store

● Análisis - Caso 1:

```
10 void lock(lock_t *mutex) {  
11     while (mutex->flag == 1),  
12     mutex->flag = 1;  
13 }
```

Thread 1

call lock()

while (flag == 1)

flag = 1;

interrupt: T1 → T2

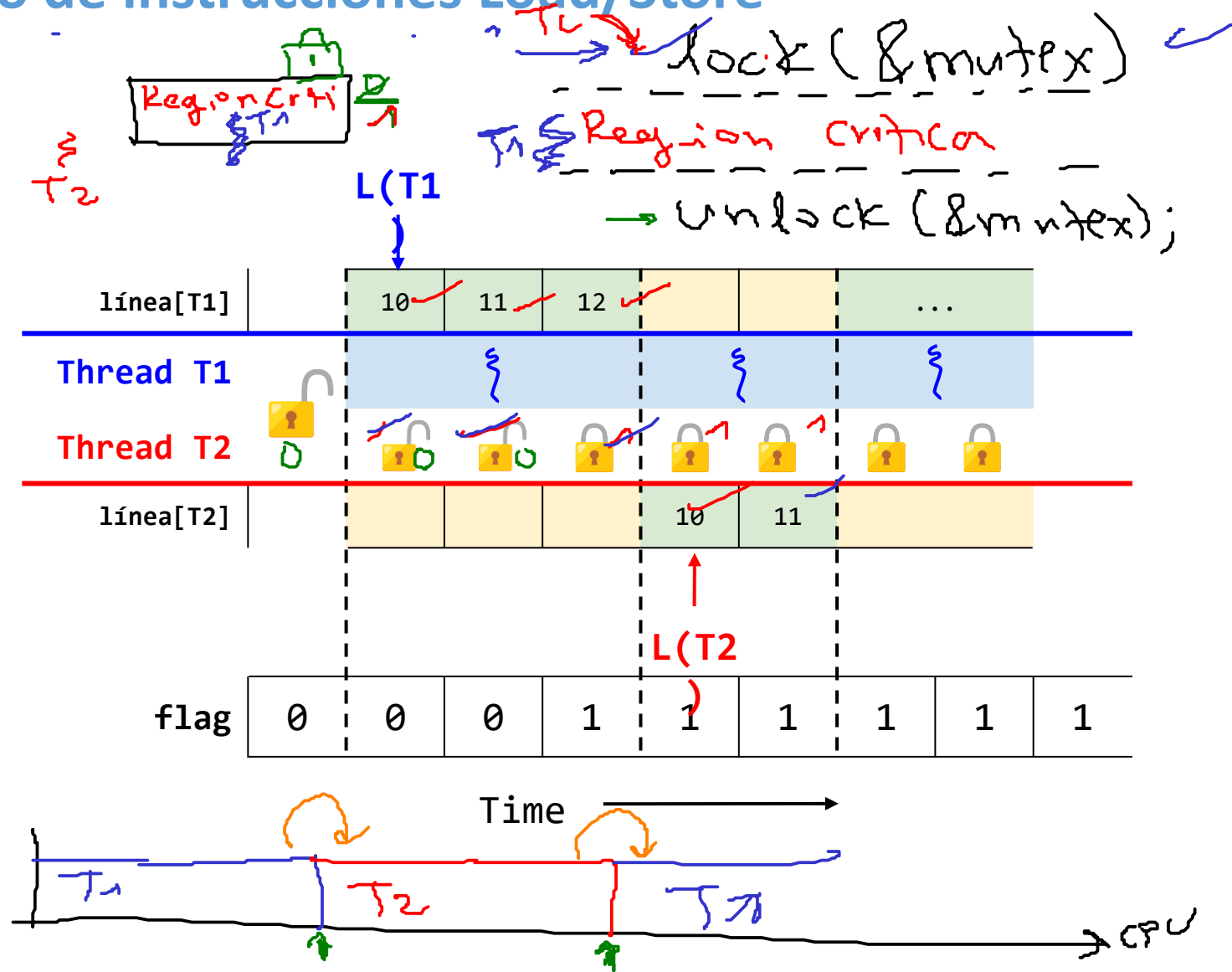
Thread 2

call lock()

while (flag == 1)

while (flag == 1)

interrupt: T2 → T1



Implementación de locks

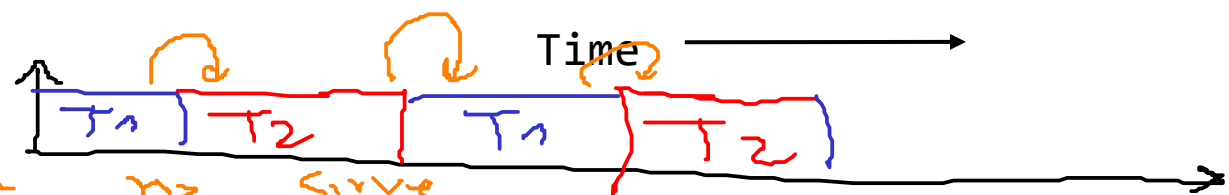
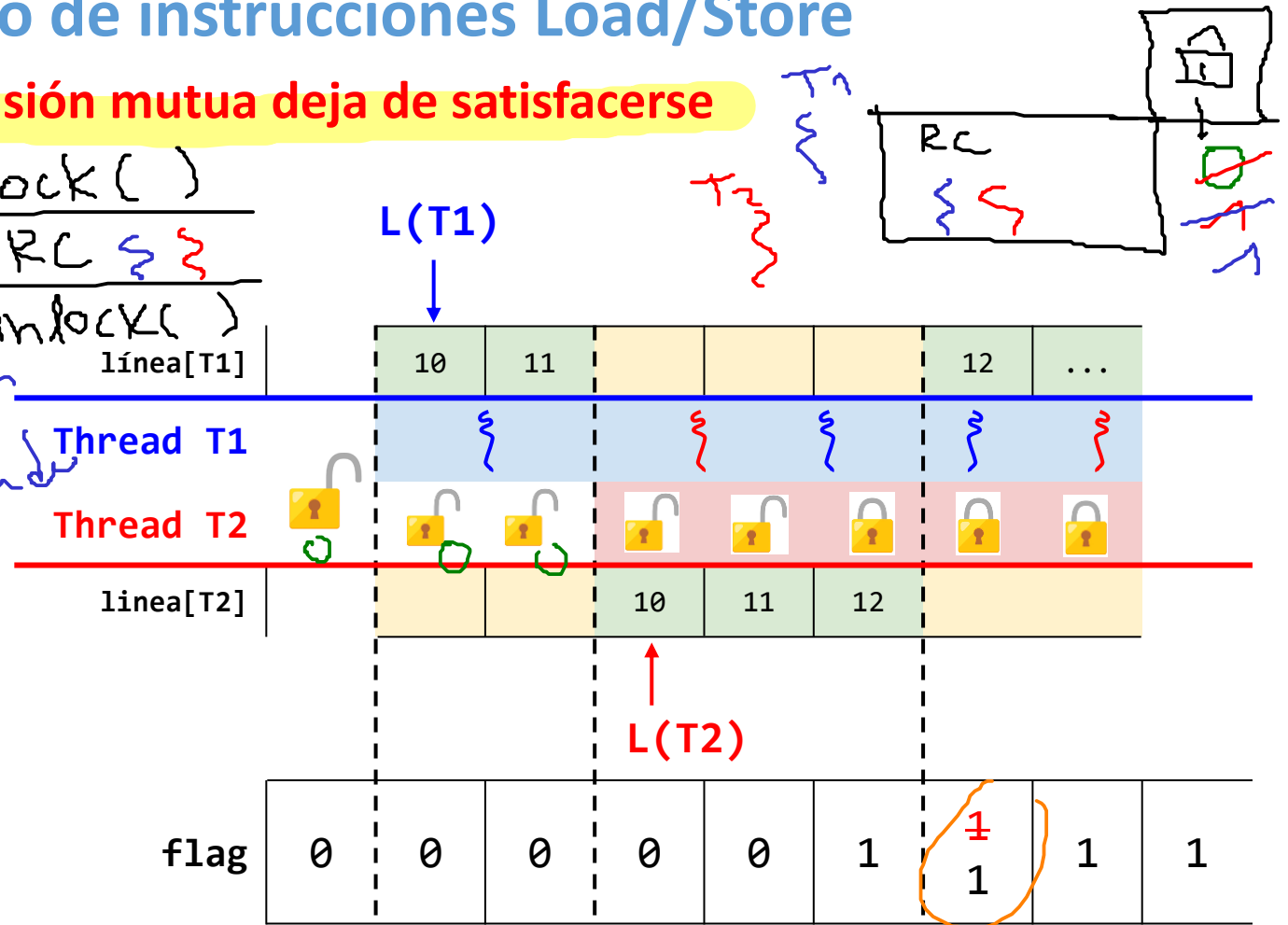
Implementación mediante el uso de instrucciones Load/Store

- **Análisis - Caso 2: Problema - La exclusión mutua deja de satisfacerse**

```
10 void lock(lock_t *mutex) {  
11     while (mutex->flag == 1);  
12     mutex->flag = 1;  
13 }
```

lock()
RC { }
unlock()
línea[T1]

Thread 1	Thread 2
call lock() ✓	
while (flag == 1) ✓	
interrupt: T1 → T2	
	call lock() ✓
	while (flag == 1) ✓
	flag = 1; ✓
	interrupt: T2 → T1
flag = 1;	



La solución como esta no sirve

Implementación de locks

Implementación mediante el uso de instrucciones Load/Store

Problema 1 - No se asegura la exclusión mutua

- Ambos hilos ingresan a la región crítica.

```

Thread 1
call lock ()
while (flag == 1)
interrupt: switch to Thread 2

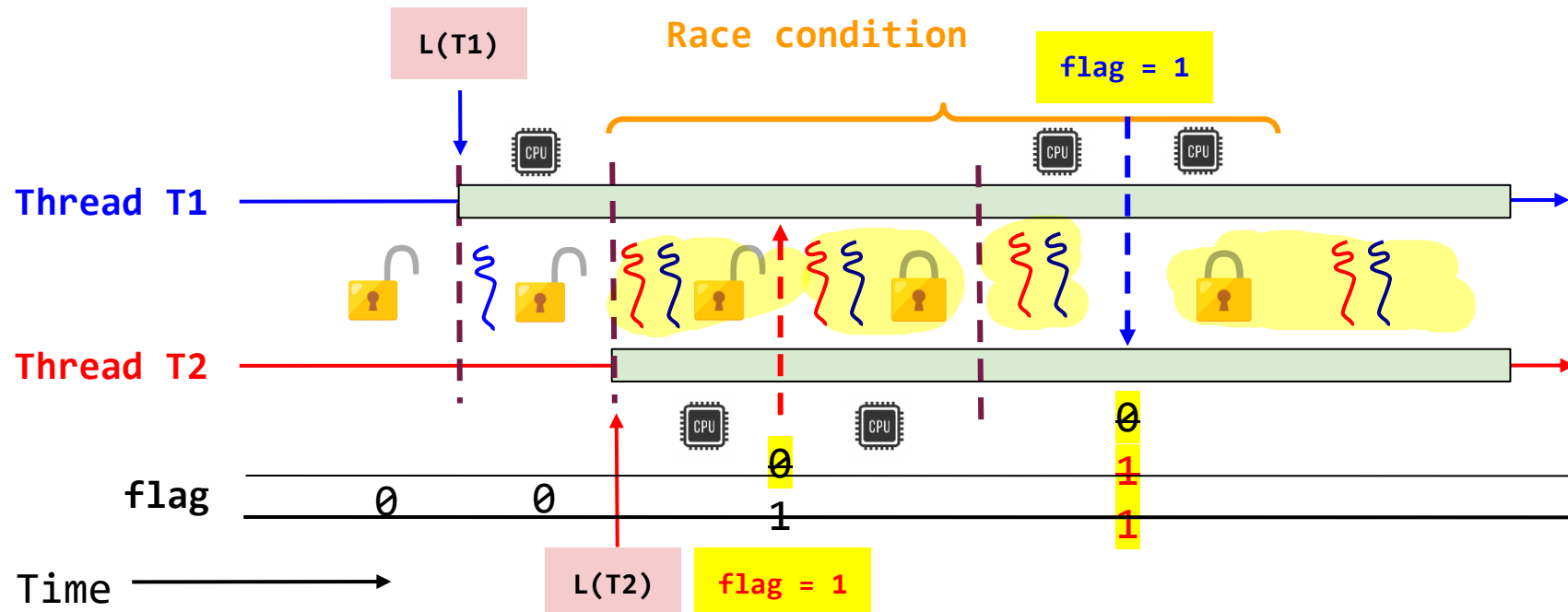
```

Thread 2

```
call lock()  
while (flag == 1)  
flag = 1;  
interrupt: switch to Thread 1
```

```
flag = 1; // set flag to 1 (too!)
```

Trace: No Mutual Exclusion



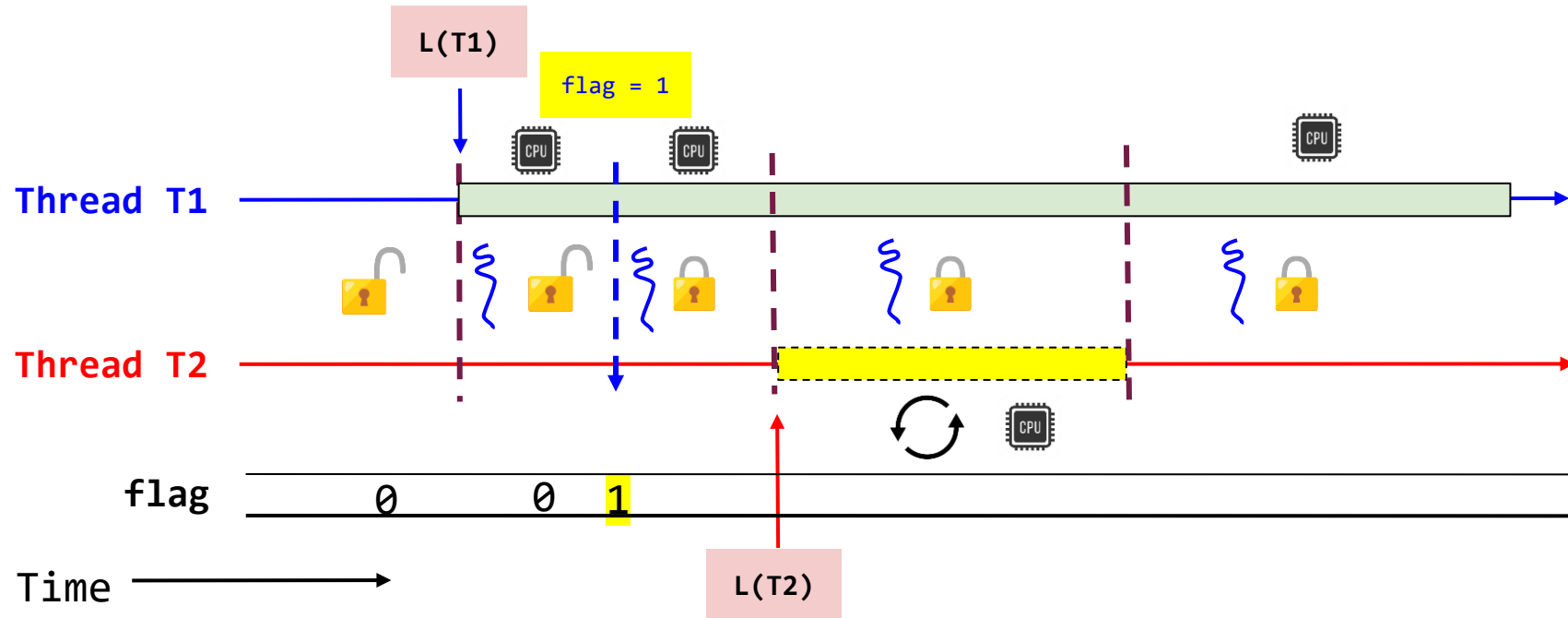
Implementación de locks

Implementación mediante el uso de instrucciones Load/Store

Problema 2 - Desempeño:

- Gasto intensivo de CPU debido a la forma como el hilo chequea la bandera.
 - **spin-waiting**: Chequeo continuo del valor de la bandera (`flag`)

```
void lock(lock_t *mutex) {  
    while (mutex->flag == 1); // spin-wait (do nothing)  
    mutex->flag = 1; // now SET it!  
}
```



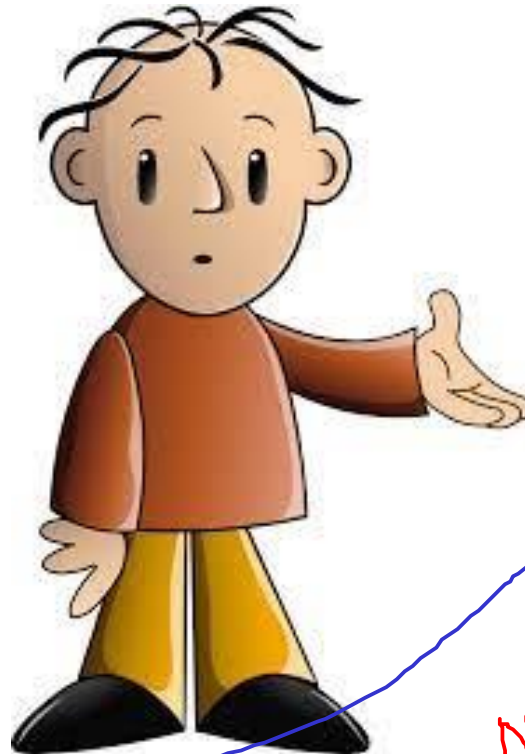
Implementación de locks

Implementación mediante el uso de instrucciones Load/Store

Conclusión:

- Una solución como la anterior (que es solo de software) no sirve.
- Se necesita soporte de hardware.

Que instrucciones
de ASM ~~asemblos~~
a la CPU
nos sirven?



instrucciones
atómicas

No se pueden interrumpir.

Implementación de locks

Instrucción test-and-set

- **Instrucciones atómicas:** Garantizan que las operaciones de lectura, modificación y escritura (read-modify-write) sean ejecutadas atómicamente (sin interrupciones).

- **Instrucción test-and-set (Intercambio atómico):**

- x86: xchg
- RISC-V: amoswap

✓ int r = testAndSet(&a, 9);
✓ r = testAndSet(&a, 10);

a [89] → 9
r [89]

Atómica

```
int testAndSet(int *old_ptr, int new) {  
    int old = *old_ptr; // fetch old value at old_ptr  
    *old_ptr = new;     // store 'new' into old_ptr  
    return old;         // return the old value  
}
```

→ xchg

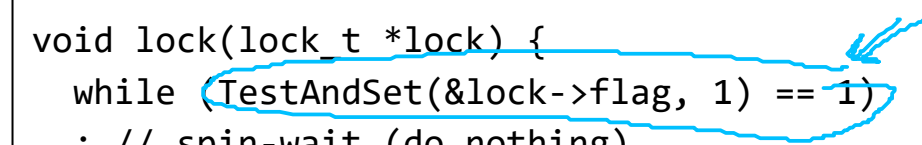
- La instrucción **test-and-set** devuelve el valor anterior y actualiza (de manera simultánea) el valor de una posición de memoria.

Implementación de locks

Implementación de un lock usando la instrucción test-and-set

- Implementación de un spinlock la instrucción **test-and-set** (xv6: [spinlock.h](#); [spinlock.c](#))

```
1  typedef struct __lock_t {
2      int flag;
3  } lock_t;
4
5  void init(lock_t *lock) {
6      // 0: lock is available, 1: lock is held
7      lock->flag = 0;
8  }
9
10 void lock(lock_t *lock) {
11     while (TestAndSet(&lock->flag, 1) == 1)
12         ; // spin-wait (do nothing)
13 }
14
15 void unlock(lock_t *lock) {
16     //lock->flag = 0;
17     TestAndSet(&lock->flag, 0);
18 }
```



Implementación de locks

Implementación de un lock usando la instrucción test-and-set

```
1 typedef struct __lock_t {
2     int flag;
3 } lock_t;
4
5 void init(lock_t *mutex) {
6     // 0 -> lock is available, 1 -> held
7     mutex->flag = 0;
8 }
9
10 void lock(lock_t *mutex) {
11     while (mutex->flag == 1) // TEST the
12         flag
13     ; // spin-wait (do nothing)
14     mutex->flag = 1; // now SET it!
15 }
16
17 void unlock(lock_t *mutex) {
18     mutex->flag = 0;
19 }
```



```
1 typedef struct __lock_t {
2     int flag;
3 } lock_t;
4
5 void init(lock_t *lock) {
6     // 0: lock is available, 1: lock is
7     held
8     TestAndSet(&lock->flag, 0);
9 }
10
11 void lock(lock_t *lock) {
12     while (TestAndSet(&lock->flag, 1) == 1)
13     ; // spin-wait (do nothing)
14 }
15
16 void unlock(lock_t *lock) {
17     TestAndSet(&lock->flag, 0);
18 }
```

Implementación de locks

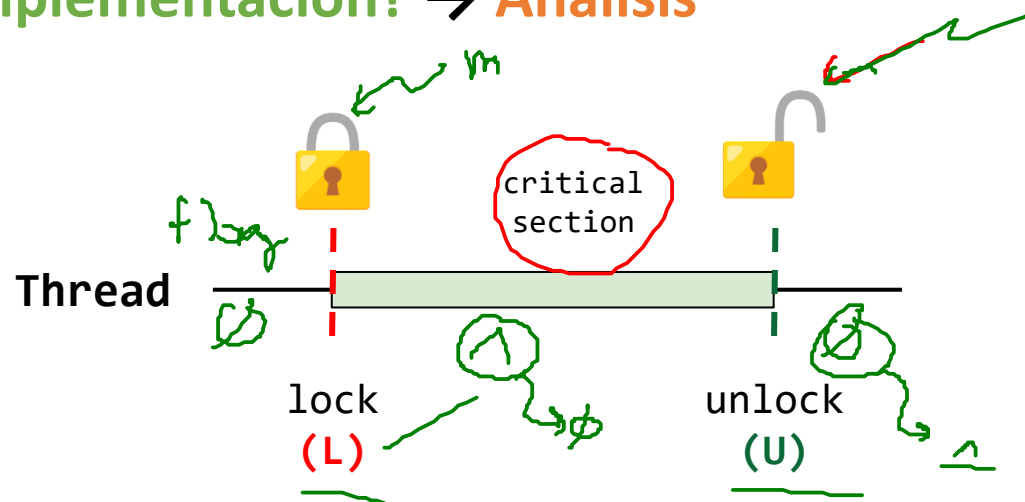
Implementación de un lock usando la instrucción test-and-set

```
void lock(lock_t *lock) {
    while (TestAndSet(&lock->flag, 1) == 1)
        ; // spin-wait (do nothing)
}
```

```
void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

¿Por qué funciona esta implementación? → Análisis

```
lock(&m);  
Critical Section  
unlock(&m);
```



Implementación de locks

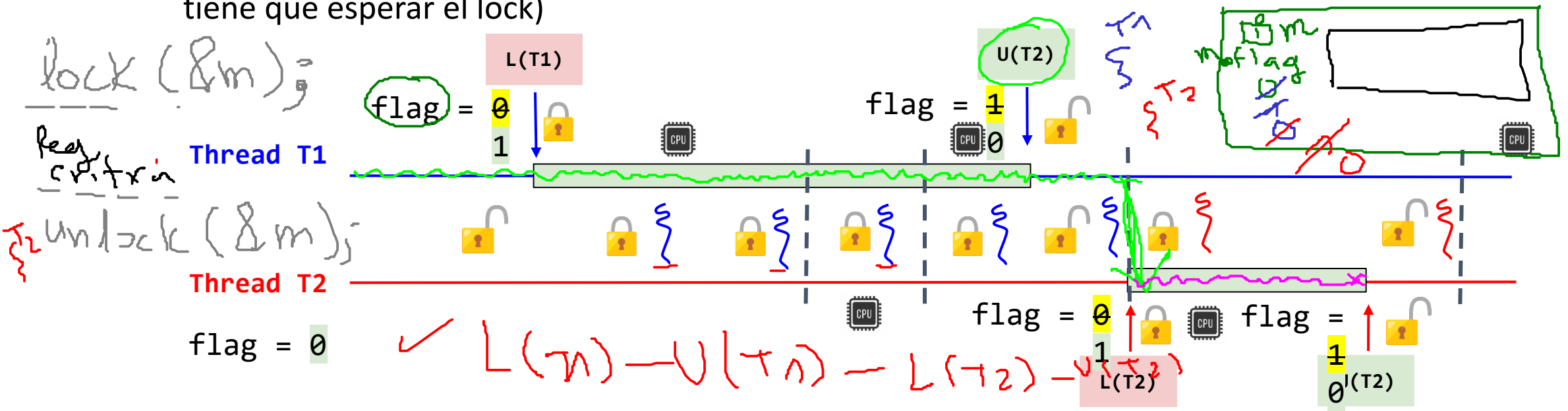
Implementación de un lock usando la instrucción test-and-set

¿Por qué funciona esta implementación? → **Análisis**

```
void lock(lock_t *lock) {  
    while (testAndSet(&lock->flag, 1) == 1)  
        ; // spin-wait (do nothing)  
}
```

```
void unlock(lock_t *lock) {  
    lock->flag = 0;  
}
```

Caso 1: Un hilo adquiere un lock y ningún otro hilo concurrente intenta adquirirlo (por lo tanto no tiene que esperar el lock)



Implementación de locks

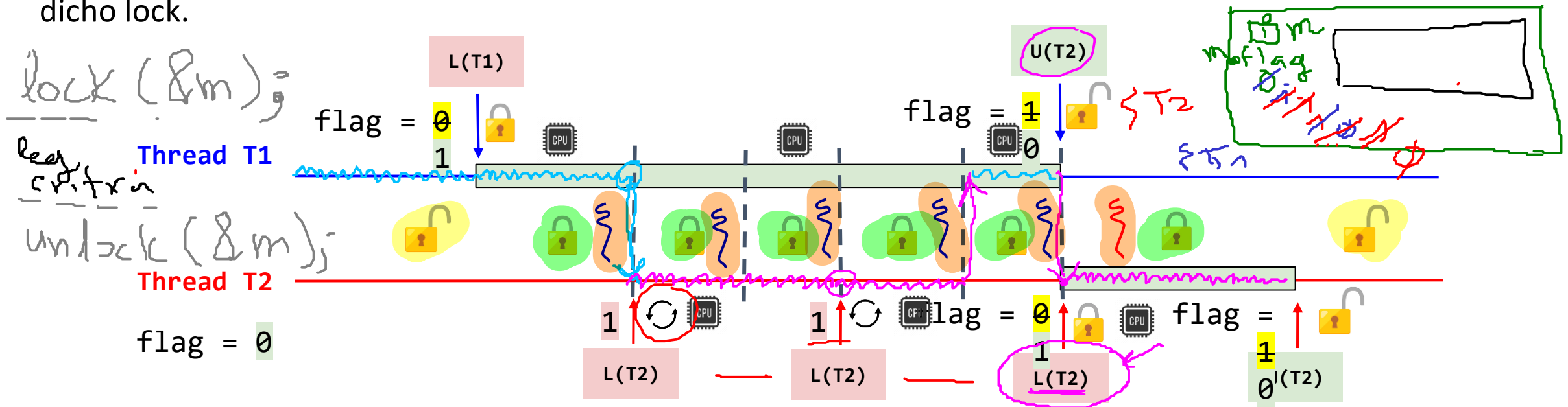
Implementación de un lock usando la instrucción test-and-set

¿Por qué funciona esta implementación? → Análisis

```
void lock(lock_t *lock) {  
    while (TestAndSet(&lock->flag, 1) == 1)  
        ; // spin-wait (do nothing)  
}
```

```
void unlock(lock_t *lock) {  
    lock->flag = 0;  
}
```

Caso 2: Un hilo adquiere un lock y una vez este ha sido adquirido, otro hilo concurrente intenta adquirir dicho lock.



Implementación de locks

Implementación de un lock usando la instrucción test-and-set

¿Por qué funciona esta implementación? → **Conclusión**

Métrica	Observaciones
Correctness (Exc. Mutua)	Si: • Solo permite que un hilo se encuentre en la región crítica. ✓
Fairness (Equidad)	No: • No proporciona acceso equitativo a todos los hilos. • Un hilo podría quedarse eternamente en el loop → Inanición
Performance (Desempeño)	No: • Alto costo: Monoprocesador. • Si el número de procesadores es comparable con el número de hilos en ejecución, el sistema trabaja bien.

Implementación de locks

Implementación de un lock usando la instrucción Compare-And-Swap

- Esta instrucción atómica actualiza una localización de memoria con un valor anterior sólo cuando este es igual al esperado. (x86: cmpxchg).
- Esta instrucción trabaja de manera similar al test-and-set pero es más poderosa.

```
int CompareAndSwap(int *ptr, int expected, int new) {  
    int original = *ptr;  
    if (original == expected)   
        *ptr = new;  
    return original;  
}
```

Handwritten notes and diagrams illustrating the function's behavior:

- Annotations in the code: `ptr` is circled in pink, `expected` has a `0` above it, and `new` has a `200` above it.
- Diagram of memory state: A vertical line represents memory. A green circle contains `0`, with a blue arrow pointing to it from the text `(0, 100);`. Below it, another green circle contains `200`, with a pink arrow pointing to it from the text `(200);`. A blue arrow points from the text `global` to the `200` circle.
- Execution flow: A pink arrow points from the `original = *ptr;` line to a box containing `100`. A blue arrow points from the `if (original == expected)` line to the same box.
- Success cases: Two orange boxes labeled "success" are shown. The first contains `CompareAndSwap(&global, 0, 200);` and the second contains `CompareAndSwap(&global, 200, 100);`. A pink arrow points from the first box to the `200` circle in the memory diagram. A blue arrow points from the second box to the `100` box.

- Análisis del pseudocódigo implementado en C: Chequea si el valor (`*ptr`):
 - Si `*ptr == esperado` → actualiza la memoria al valor new. ✓
 - Si `*ptr != esperado` → retorna el valor actual. ✓

Implementación de locks

Implementación de un lock usando la instrucción Compare-And-Swap

```
void lock(lock_t *lock) {  
    while (CompareAndSwap(&lock->flag, 0, 1) == 1)  
        ; // spin  
}
```

```
int CompareAndSwap(int *ptr, int expected, int  
new) {  
    int original = *ptr;  
    if (original == expected)  
        *ptr = new;  
    return original;  
}
```

```
void unlock(lock_t *lock) {  
    lock->flag = 0;  
}
```

lock

unlock

Implementación de locks

Implementación de un lock usando la instrucción Compare-And-Swap

lock

```
void lock(lock_t *lock) {
    while (CompareAndSwap(&lock->flag, 0, 1) == 1)
        ; // spin
}
```

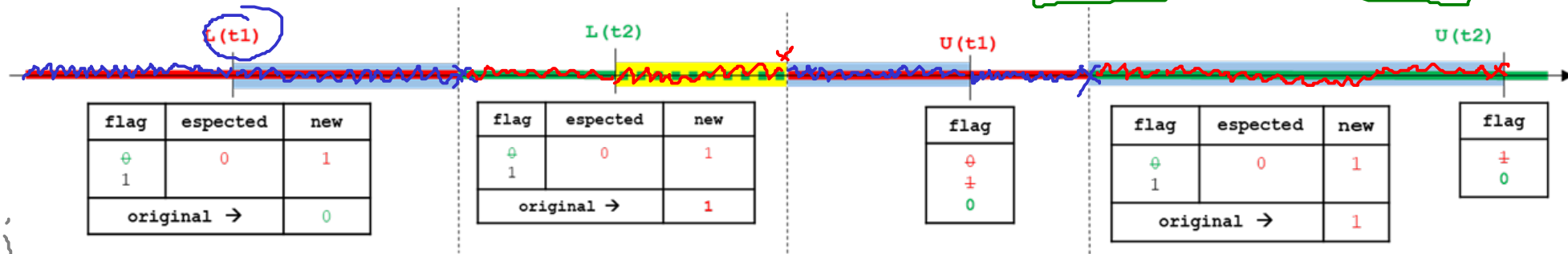
unlock

```
void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

CPU
T₂

M.flag RC
0 10

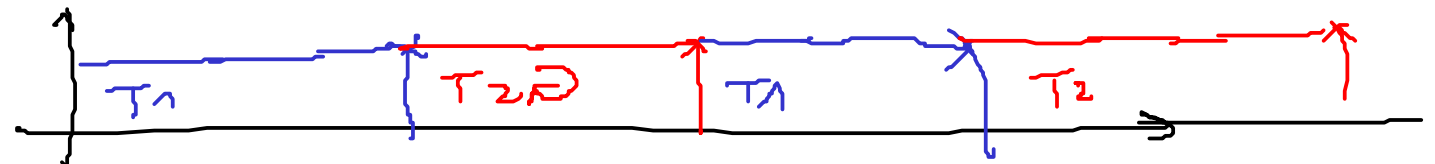
thread 1 (t1)
thread 2 (t2)



lock(&m);

Region critical

unlock(&m);



Implementación de locks

Video de clase

Implementación con instrucciones LL&SC (load linked – store conditional)

LL (Load linked): Opera como una instrucción load típica trayendo simplemente un valor de memoria y poniéndolo en un registro.

```
int LoadLinked(int *ptr) {  
    return *ptr;  
}
```

SC (Store conditional): El almacenamiento solo funciona si no se ha realizado otro almacenamiento previo en el mismo lugar:

- **Éxito:** Devuelve **1** y actualiza el valor de ***ptr** a new.
- **Falla:** Retorna **0** sin actualizar la memoria.

```
int StoreConditional(int *ptr, int value) {  
    if (no update to *ptr since LoadLinked to this address) {  
        *ptr = value;  
        return 1; // success!  
    }  
    else {  
        return 0; // failed to update  
    }  
}
```

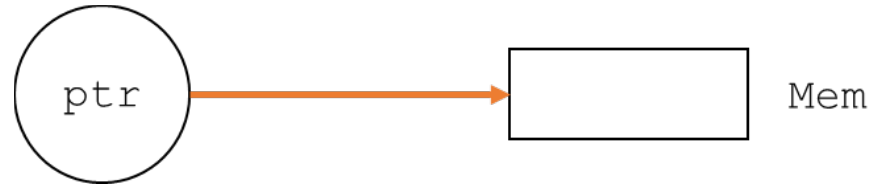
Implementación de locks

Video de clase

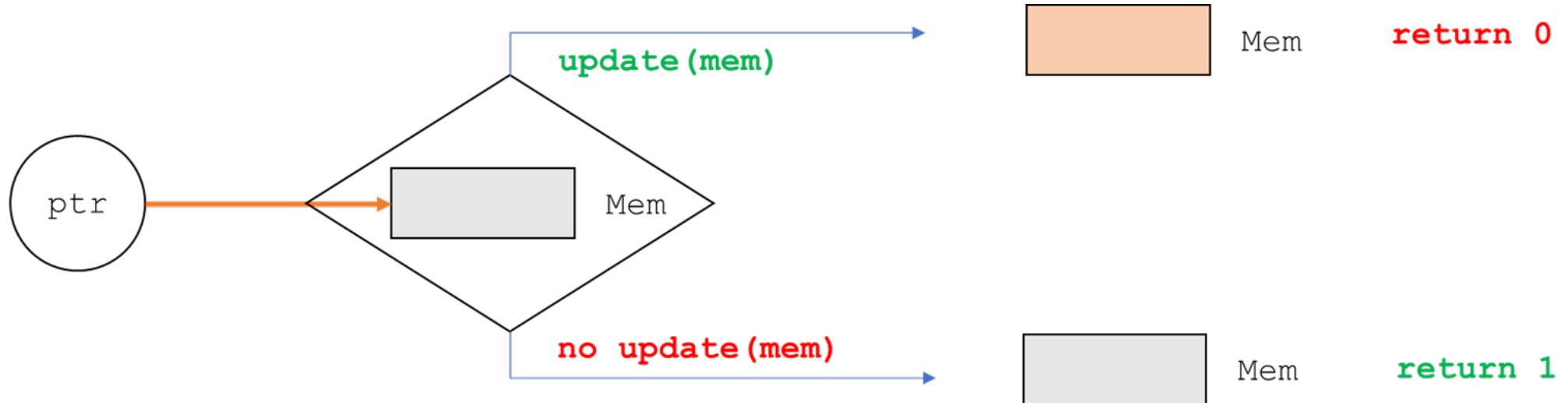
Implementación con instrucciones LL&SC (load linked – store conditional)

no update to *ptr since LoadLinked to this address – Que es?

```
int LoadLinked(int *ptr)
```



```
int StoreConditional(int *ptr, int value)
```



Implementación de locks

Videos clase

Implementación con instrucciones LL&SC (load linked – store conditional)

Implementación del lock

```
void lock(lock_t *lock) {
    while (1) {
        while (LoadLinked(&lock->flag) == 1)
            ; // spin until it's zero
        if (StoreConditional(&lock->flag, 1) == 1)
            return; // if set-it-to-1 was a success: all done
                    // otherwise: try it all over again
    }
}
```

```
void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

Implementación de locks

Video class

Implementación con instrucciones LL&SC (load linked – store conditional)

¿Por qué funciona? - Análisis

```
void lock(lock_t *lock) {  
    while (1) {  
        while (LoadLinked(&lock->flag) == 1);  
        if (StoreConditional(&lock->flag, 1) == 1)  
            return;  
    }  
}
```

```
int LoadLinked(int *ptr) {  
    return *ptr;  
}
```

```
int StoreConditional(int *ptr, int value) {  
    if (no update to *ptr since LoadLinked to this address) {  
        *ptr = value;  
        return 1; // success!  
    }  
    else {  
        return 0; // failed to update  
    }  
}
```

```
void unlock(lock_t *lock) {  
    lock->flag = 0;  
}
```

Implementación de locks

Implementación con instrucciones LL&SC (load linked – store conditional)

¿Por qué funciona? - Análisis

```
void lock(lock_t *lock) {  
    while (1) {  
        while (LoadLinked(&lock->flag) == 1);  
        if (StoreConditional(&lock->flag, 1) == 1)  
            return;  
    }  
}
```

```
void unlock(lock_t *lock) {  
    lock->flag = 0;  
}
```

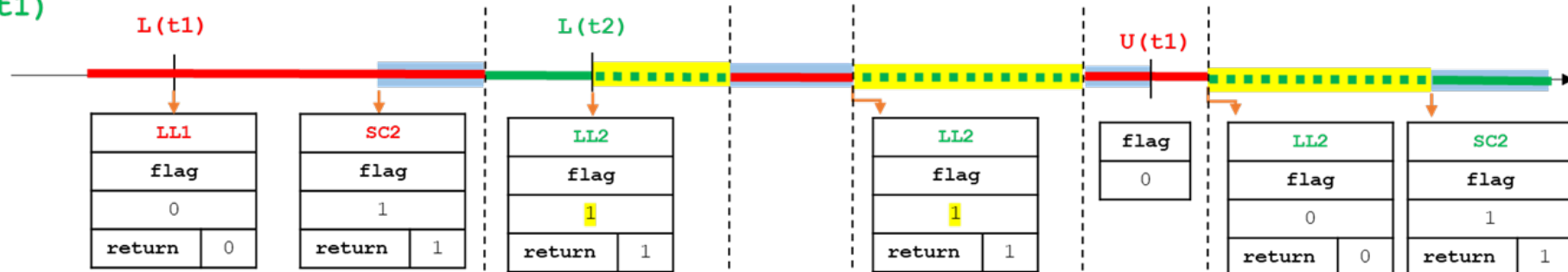
Videos de clase

Caso 1: Secuencia LL1 - SC1 - LL2 - SC2

thread 1 (t1)

thread 2 (t2)

flag = 0



Implementación de locks

Implementación con instrucciones LL&SC (load linked – store conditional)

¿Por qué funciona? - Análisis

```
void lock(lock_t *lock) {  
    while (1) {  
        while (LoadLinked(&lock->flag) == 1);  
        if (StoreConditional(&lock->flag, 1) == 1)  
            return;  
    }  
}
```

```
void unlock(lock_t *lock) {  
    lock->flag = 0;  
}
```

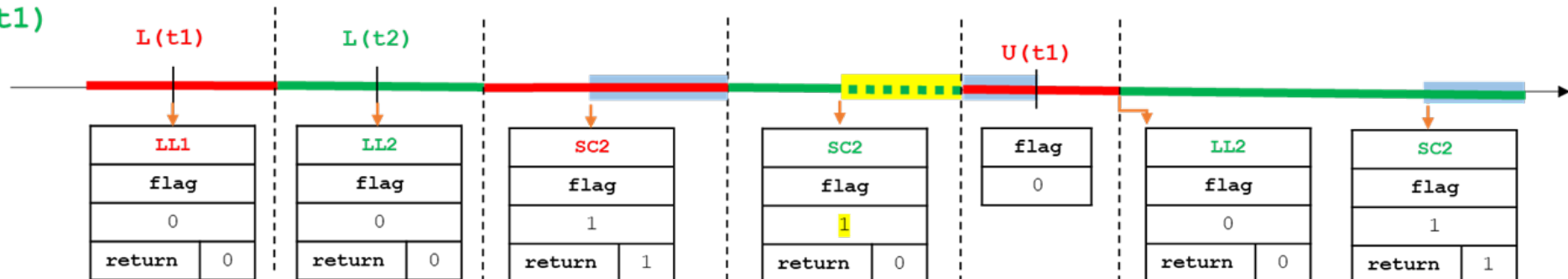
Video Case

Caso 2: Secuencia LL1 - LL2 - SC2 - SC2

thread 1 (t1)

thread 2 (t2)

flag = 0



Implementación de locks

Video clase

Implementación usando la instrucción fetch-and-add:

Incrementa posición de memoria, retornando (de manera simultánea) el valor antiguo:

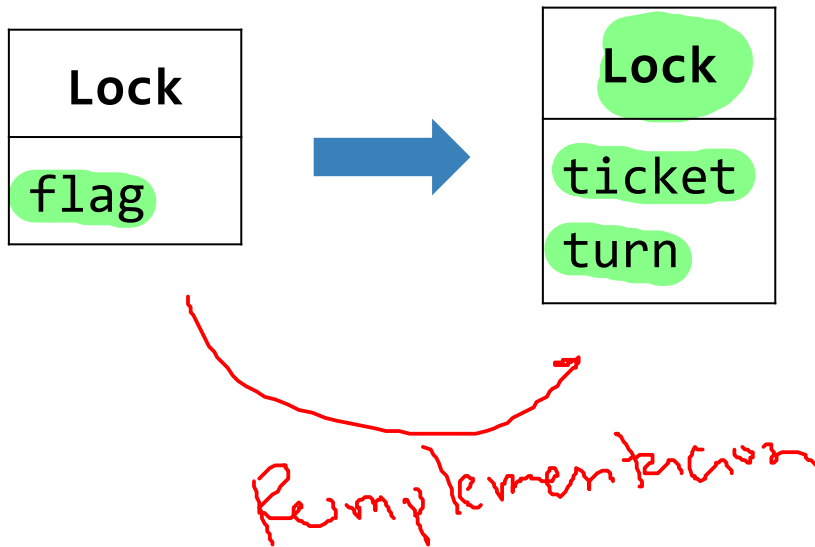
```
int FetchAndAdd(int *ptr) {  
    int old = *ptr;  
    *ptr = old + 1;  
    return old;  
}
```

Mellon-Crummey y Scott usaron esta instrucción para construir un mecanismo de acceso a regiones críticas conocido como ticket lock.

Implementación de locks

Ticket-lock usando Fetch-and-Add

En vez de un solo valor (**flag**) esta instrucción usa un **ticket** y un **turn** para construir el lock.



```
typedef struct __lock_t {
    int ticket;
    int turn;
} lock_t;

void lock_init(lock_t *lock) {
    lock->ticket = 0;
    lock->turn = 0;
}

void lock(lock_t *lock) {
    int myturn = FetchAndAdd(&lock->ticket);
    while (lock->turn != myturn) {
        ; // spin
    }
}

void unlock(lock_t *lock) {
    lock->turn = lock->turn + 1;
}
```

Implementación de locks

Ticket-lock - Análisis

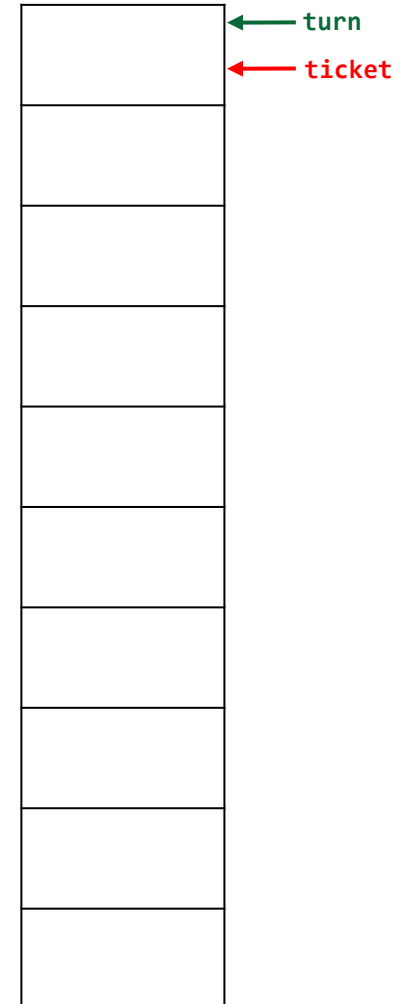
```
typedef struct __lock_t {
    int ticket;
    int turn;
} lock_t;

void lock_init(lock_t *lock) {
    lock->ticket = 0;
    lock->turn = 0;
}

void lock(lock_t *lock) {
    int myturn = FetchAndAdd(&lock->ticket);
    while (lock->turn != myturn) {
        ; // spin
    }
}

void unlock(lock_t *lock) {
    lock->turn = lock->turn + 1;
}
```

	ticket	turn	R - Run S - Spin State (R/S)
lock_init	0	0	---
A lock()			
B lock()			
C lock()			
A unlock()			
B			
A lock()			
B unlock()			
C			
C unlock()			
A			
A unlock()			
C lock()			



Implementación de locks

Ticket-lock - Análisis

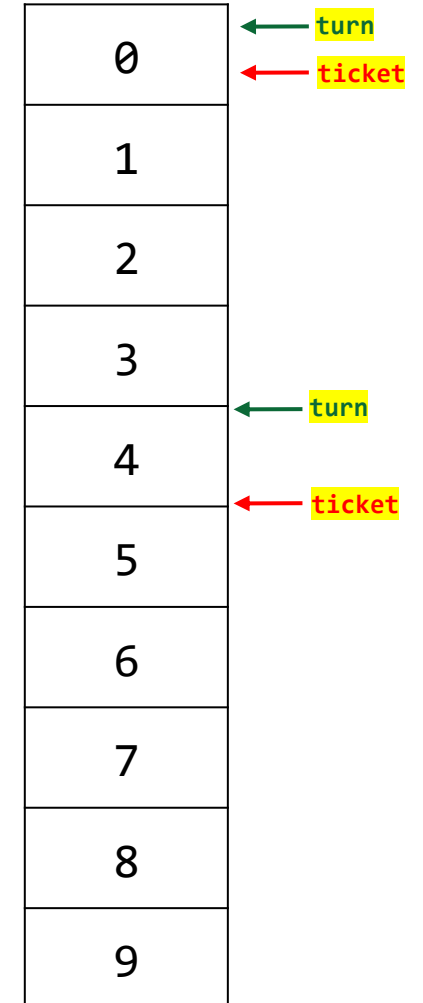
```
typedef struct __lock_t {
    int ticket;
    int turn;
} lock_t;

void lock_init(lock_t *lock) {
    lock->ticket = 0;
    lock->turn = 0;
}

void lock(lock_t *lock) {
    int myturn = FetchAndAdd(&lock->ticket);
    while (lock->turn != myturn) {
        ; // spin
    }
}

void unlock(lock_t *lock) {
    lock->turn = lock->turn + 1;
}
```

	ticket	turn	R - Run S - Spin State (R/S)
lock_init	0	0	---
A lock()	1	0	R
B lock()	2	0	S(1)
C lock()	3	0	S(2)
A unlock()	3	1	---
B			R
A lock()	4	1	S(3)
B unlock()	4	2	---
C			R
C unlock()	4	3	---
A			R
A unlock()	4	4	---
C lock()	5	4	R



Implementación de locks

Resumen

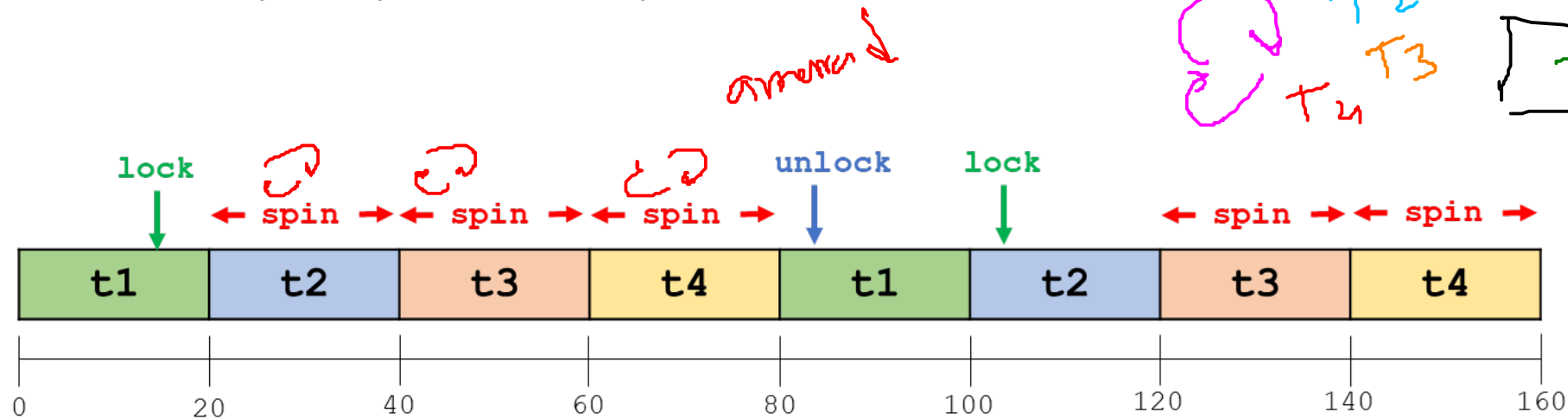
- Aspectos claves de los spinlocks:
 - Simples de **implementar**
 - El hilo que está en espera activa **desperdicia todo el quantum de tiempo** haciendo **nada** (solo chequea la variable). *(Hilo amarrado en el while)*
 - Gran gasto de CPU, esto los hace costosos.
- Los **spinlocks** (y deshabilitar interrupciones en sistemas con un única CPU) son mecanismos de sincronización primitivos.
 - Son empleados para crear construcciones de sincronización de alto nivel
- Para evitar el gran gasto de CPU se requiere **asistencia del sistema operativo**.

¿Cómo evitar gasto en la CPU?

¿Cuál es el problema con los spinlocks?

- Mientras un hilo espera el lock **gasta tiempo de uso del procesador chequeando un valor** que no sabe cuándo cambiará (**gasto de CPU**).
- Entre más hilos (**N hilos**) más ciclos de CPU son gastados (**N - 1**), de modo que el problema empeora.

```
void lock(lock_t *lock) {  
    while (TestAndSet(&lock->flag, 1) != 1);  
}
```



Idea clave ¿Cómo evitar el spinning?

¿Cómo desarrollar un lock que no gaste innecesariamente tiempo de la CPU iterando?

¿Cómo evitar gasto en la CPU?

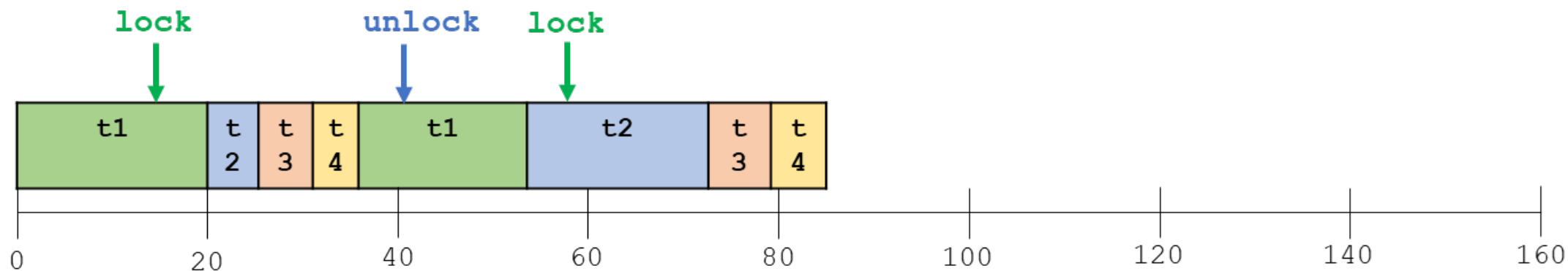
Yield en vez de spin

Cuando se requiera realizar espera activa, simplemente libera voluntariamente la CPU (llamado al sistema **yield**).

```
void lock(lock_t *lock) {  
    while (TestAndSet(&lock->flag, 1) == 1);  
}
```



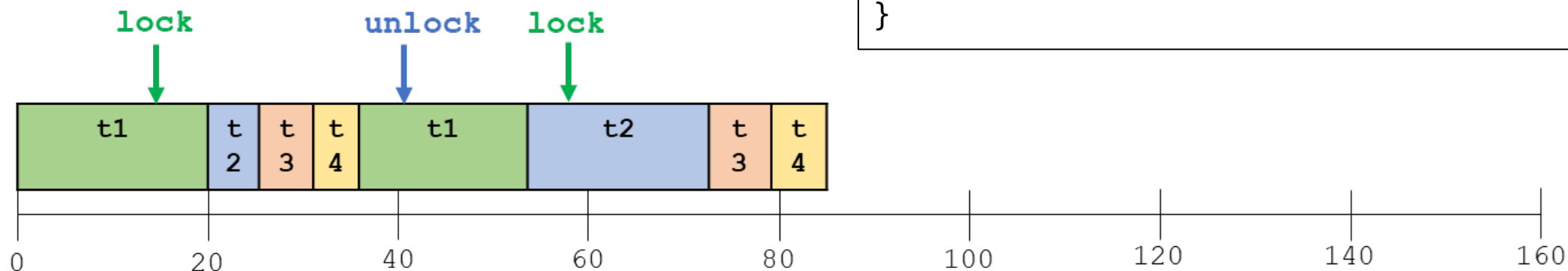
```
void lock(lock_t *lock) {  
    while (TestAndSet(&lock->flag, 1) == 1)  
        yield(); // give up the CPU  
}
```



¿Cómo evitar gasto en la CPU?

Yield en vez de spin

- El sistema operativo mueve el hilo de estado running a ready.
- Aún se gasta tiempo en los cambios de contexto.
- Puede existir inanición.



```
typedef struct __lock_t {
    int flag;
} lock_t;

void init(lock_t *lock) {
    lock->flag = 0;
}

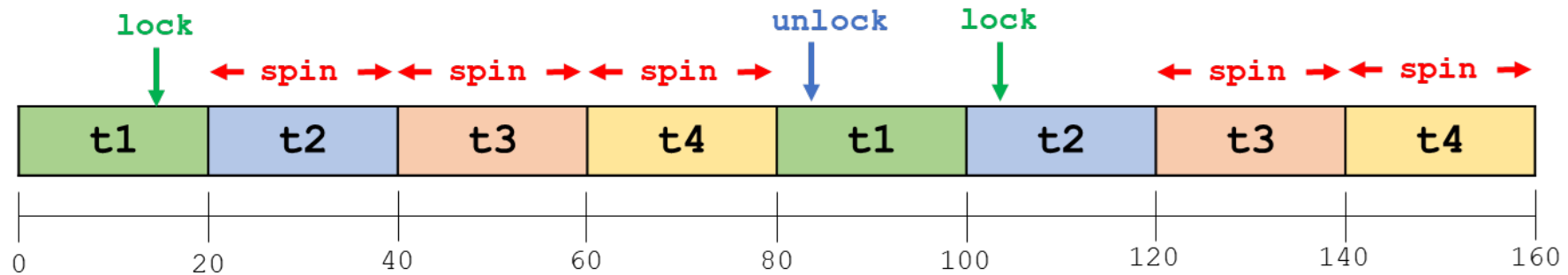
void lock(lock_t *lock) {
    while (TestAndSet(&lock->flag, 1) == 1)
        yield();
}

void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

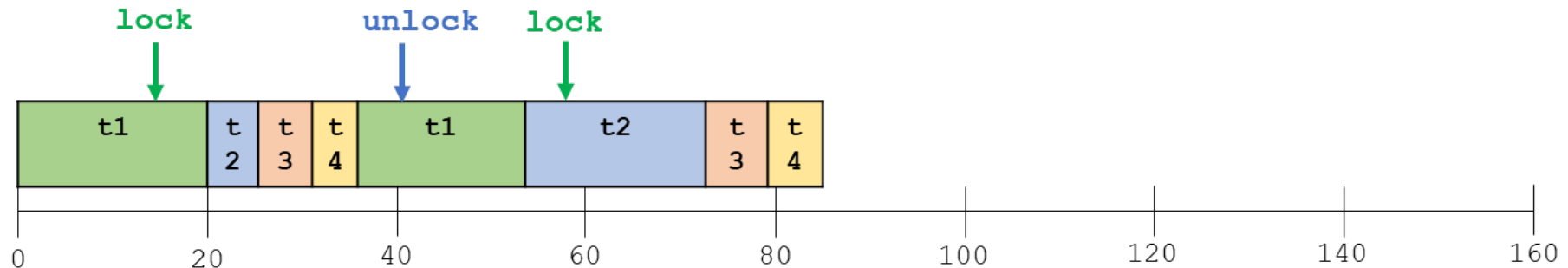
¿Cómo evitar gasto en la CPU?

Comparación

Sin `yield()`: o $(\text{threads} * \text{time_slice})$



Con `yield()`: o $(\text{threads} * \text{context_switch})$



Queue locks

Problemas con las colas

 Video

Acá vamos...

Queue locks

Uso de colas

← Video

Idea:

- Poner los hilos en una cola (queue).
- Indicarle al kernel que no agende (don't schedule) los hilos en la cola.
- Al desbloquear (unlock), indicar al kernel que puede ejecutar subprocesos nuevamente.
- Llamadas:
 - **park()**: El hilo que llama la función va a dormir (sleep state).
 - **unpark(threadID)**: Despierta un hilo en particular.

Queue locks

Uso de colas

```
typedef struct __lock_t {  
    int flag;  
    int guard;  
    queue_t *q;  
} lock_t;
```

```
void lock_init(lock_t *m) {  
    m->flag = 0;  
    m->guard = 0;  
    queue_init(m->q);  
}
```

```
void lock(lock_t *m) {  
    while (TestAndSet(&m->guard, 1) == 1)  
        ; //acquire guard lock by spinning  
    if (m->flag == 0) {  
        m->flag = 1; // lock is acquired  
        m->guard = 0;  
    } else {  
        queue_add(m->q, getpid());  
        m->guard = 0;  
        park();  
    }  
}
```

```
void unlock(lock_t *m) {  
    while (TestAndSet(&m->guard, 1) == 1)  
        ; //acquire guard lock by spinning  
    if (queue_empty(m->q))  
        m->flag = 0; // let go of lock; no one wants it  
    else  
        unpark(queue_remove(m->q)); // hold lock  
                                     // (for next thread!)  
    m->guard = 0;  
}
```

Referencias

- <https://adit.io/posts/2013-05-15-Locks,-Actors,-And-STM-In-Pictures.html>
- https://link.springer.com/chapter/10.1007/978-1-4842-4398-5_5
- <https://preshing.com/20150316/semaphores-are-surprisingly-versatile/>
- <https://jvns.ca/teach-tech-with-cartoons/>
- <https://code-cartoons.com/>
- <https://medium.com/basecs>
- <https://xkcd.com/>
- <https://www.qwantz.com/>
- <https://pbfcomics.com/>
- <http://turnoff.us/>
- <http://turnoff.us/geek/arduino-project/>
- <https://prepinsta.com/operating-systems/critical-section-problem/>
- <https://jakegut.com/posts/sg-os/>
- <https://realpython.com/intro-to-python-threading/>
- https://github.com/Marcombo/Programacion_de_servicios_y_procesos_en_Python
- <https://www.cs.cornell.edu/courses/cs3410/2012sp/>
- <https://www.cs.cornell.edu/courses/cs3410/2012sp/lecture/25-sync-i-g.pdf>
- <https://www.cs.cornell.edu/courses/cs3410/2012sp/schedule.html>

Referencias

- <https://csapp.cs.cmu.edu/>
- <https://applied-programming.github.io/Operating-Systems-Notes/5-Memory-Management/>
- <http://www.cs.cmu.edu/~213/>
- <https://web.eecs.umich.edu/~akamil/teaching/sp04/>
- <https://courses.engr.illinois.edu/cs241/sp2014/>
- <https://emeryberger.com/teaching/grad-systems/>
- <https://oslab.kaist.ac.kr/courseware/>
- <https://oslab.kaist.ac.kr/ee488-fall-2020/>
- <https://github.com/JoeCameron1/CSC369-OperatingSystems-Assignments/tree/master/Assignment2>
- <http://www.cs.toronto.edu/~ylzhang/csc258/>
- <https://oslab.kaist.ac.kr/ee415-spring-2022/>
- <https://prepinsta.com/operating-systems/multiprocessor-operating-systems/>
- <https://www.iitk.ac.in/esc101/05Aug/tutorial/essential/threads/definition.html>
- <https://www.quora.com/What-are-some-best-examples-of-multithreaded-applications>
- <https://cpentalk.com/986/examples-applications-benefit-directly-ability-throughput>
- <https://www.iitk.ac.in/esc101/05Aug/tutorial/essential/threads/definition.html>
- <https://realpython.com/python-concurrency/>

Referencias

- <https://developers.google.com/web/updates/2018/09/inside-browser-part1>
- <https://developers.google.com/web/updates/2018/09/inside-browser-part2>
- <https://www.informit.com/articles/article.aspx?p=25075&seqNum=3>
- https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/4_Threads.html
- <https://realpython.com/python-sockets/>
- <https://realpython.com/intro-to-python-threading/>
- <https://realpython.com/python-concurrency/>
- <https://www.geeksforgeeks.org/socket-programming-multi-threading-python/>
- <https://eecs485staff.github.io/p4-mapreduce/threads-sockets.html>
- <https://prepinsta.com/operating-systems/inter-process-communication/>
- <https://applied-programming.github.io/Operating-Systems-Notes/6-Inter-Process-Communication/>
- <https://applied-programming.github.io/Operating-Systems-Notes/3-Threads-and-Concurrency/>
- <https://www.iitk.ac.in/esc101/05Aug/tutorial/essential/threads/definition.html>
- <https://singa.apache.org/>
- http://web.mit.edu/java_v1.0.2/www/tutorial/java/threads/deadlock.html
- <http://www.cs.fsu.edu/~baker/realtime/restricted/notes/philos.html>
- <https://www2.seas.gwu.edu/~mfeldman/papers/portable-diners.html>
- https://www.kalasim.org/examples/dining_philosophers/
- <http://people.cs.aau.dk/~ask/Undervisning/MVP/html/deadlock.pdf>

Referencias

- https://github.com/fab327/JavaFX_DiningPhilosophers
- <https://github.com/djunderw/dining-philosophers>
- <https://www.geeksforgeeks.org/dining-philosopher-problem-using-semaphores/>
- https://rosettacode.org/wiki/Dining_philosophers
- <https://www.seas.upenn.edu/~cse39904/homework/hw06.pdf>
- <https://www.seas.upenn.edu/~cse39904/>
- <https://www.seas.upenn.edu/~cse39904/notes.html>
- <https://github.com/smonn/hungrythinkers>
- <https://github.com/jnafolayan/dining-philosopher>
- <https://jnafolayan.github.io/dining-philosopher/>
- https://github.com/ZzCalvinzZ/dining_philosophers_web_workers
- <https://dev.to/vunderkind/the-dining-philosophers-problem-or-your-introduction-to-thinking-in-computer-science-8ii>
- https://github.com/lievi/dining_philosophers
- https://www.wikiwand.com/en/Dining_philosophers_problem
- <https://adit.io/posts/2013-05-11-The-Dining-Philosophers-Problem-With-Ron-Swanson.html>
- <https://github.com/tpn/pdfs>
- <https://jakegut.com/posts/sg-os/>